Supervyse OPF IPMI Feature Configuration Guide

Revision 2.7

2/2025

Copyright (c) 2025, All Rights Reserved. Insyde Software.

No part of this publication may be reproduced, transmitted, transcribed, stored in a retrieval system, or translated into any language or computer language, in any form, or by any means, electronic, mechanical, magnetic, optical, chemical, manual or otherwise, without the prior written permission of Insyde Software Corp.

Disclaimer

Insyde Software provides this document and the programs "as is" without warranty of any kind, either expressed or implied, including, but not limited to, the implied warranties of merchantability or fitness for a particular purpose.

This document could contain technical inaccuracies or typographical errors. Changes are periodically made to the information herein; these changes will be incorporated in future revisions of this document. Insyde Software is under no obligation to notify any person of the changes.

The following trademarks are used in this document:

Insyde and InsydeH2O are registered trademarks or trademarks of Insyde Software. All other trademarks or trade names are property of their respective holders.

Contents

1. Channel Table Configuration 7
   1.1. Introduction 7
   1.2. Design and Restriction 8
   1.3. Configuration 9
      1.3.1. config.xml 9
         1.3.1.1. config.xml tag description 12
         1.3.1.2. Interface specific information (Legacy)(Only apply to codes existing before G3) 13
            1.3.1.2.1. IPMB Interface 13
            1.3.1.2.2. KCS Interface 14
            1.3.1.2.3. LAN Interface 14
            1.3.1.2.4. SMM Interface 14
      1.3.2. How to use channel table on G3/G4 platform 14
      1.3.3. channel_config.json file content generated from the above exampleThe following shows the very channel_config.json generated from the above config.xml on an Intel BHS platform. 14
      1.3.4. channel_access.json 17
         1.3.4.1. channel_access.json example 18
         1.3.4.2. Tag explanation 18
2. In-band Channel Service Porting 20
   2.1. BT Channel 20
      2.1.1. Introduction 20
      2.1.2. Configuration 20
         2.1.2.1. BIOS 20
         2.1.2.2. kernel 20
         2.1.2.3. platform 20
            2.1.2.3.1. system channel service 20
            2.1.2.3.2. IPMI channel table 21
   2.2. SSIF Channel 22
      2.2.1. Introduction 22
      2.2.2. Configuration 22
         2.2.2.1. BIOS 22
         2.2.2.2. kernel 22
         2.2.2.3. platform 22
            2.2.2.3.1. system channel service 22
            2.2.2.3.2. IPMI channel table 23
   2.3. IPMB Channel 24
      2.3.1. Introduction 24
      2.3.2. Configuration 24
         2.3.2.1. Platform 24
            2.3.2.1.1. Channel table 24
3. FRU Configuration 26
   3.1. Introduction 26
   3.2. Configuration 27
      3.2.1. How to view and manage FRU data in EEPROM 28
         3.2.1.1. Update FRU Data in EEPROM 29
      3.2.2. Baseboard FRU (ID 0) configuration 31
         3.2.2.1. FRU data source from EEPROM or regular file 32
         3.2.2.2. Predefined FRU file 33
            3.2.2.2.1. Predefined FRU file path 33
            3.2.2.2.2. ipmidcfg recipe configuration 34
         3.2.2.3. Verification 35

3.2.3. How to add a new FRU ID to FRU Table 39
3.2.3.1. Add the mapping between FRU ID and I2Cbus-address 40
3.2.3.2. Add FRU SDR 41
3.2.3.3. Verification 42

4. PEF & Alert Configuration 45
4.1. Introduction 45
4.2. Configuration 45
4.2.1. How to configure default PEF alert function 46
4.2.2. Example 1 (Threshold sensor and alert with Lan) 47
4.2.2.1. Configuration in config.xml 47
4.2.2.2. Verification 50
4.2.3. Example 2 (Discrete sensor and alert with SMTP mail) 54
4.2.3.1. Configuration in config.xml 54
4.2.3.2. Verification 58

5. Device ID Configuration 62
5.1. Introduction 62
5.2. Configuration 62
5.2.1. Firmware Revision & Auxiliary Firmware Revision 62
5.2.2. Device ID, Device Revision, Additional Device Support, Manufacturer ID & Product ID 63
5.3. Verification 67

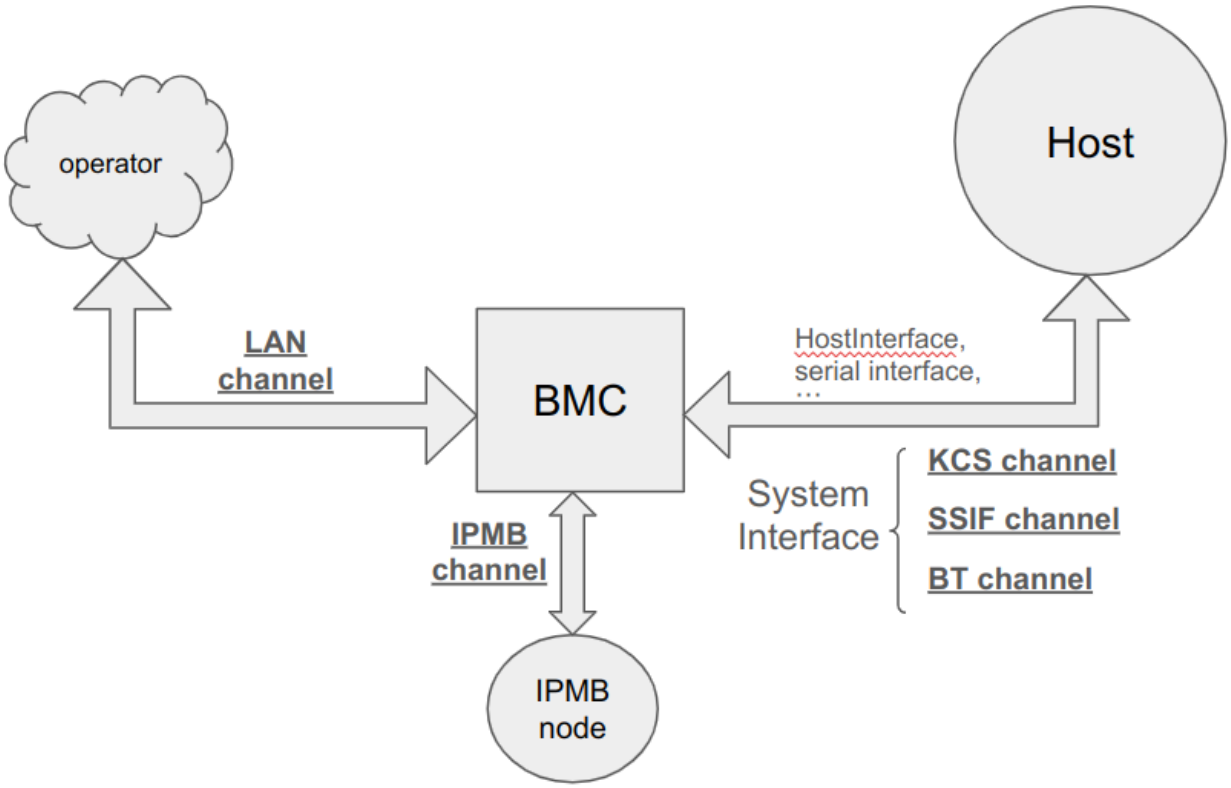
Revision History

Revision Number	Date	Author	Reviewer	Summary
1.0	2022-09-07	-	-	RV22.11 Release Document
1.1	2023-04-28	Roy Kuo	-	update for Nvidia platform
1.2	2023-05-10	Roy Kuo	-	update SDR for FRU
1.3	2023-06-13	Robert Shih	-	update BT channel and SSIF channel customization
1.4	2023-08-07	Will Xiao		update detailed network setting note
1.5	2023-09-08	Roy Kuo	-	update pre-defined FRU file
1.6	2023-11-07	Roy Kuo	-	Supervyse OPF IPMI OEM Commnad Customization Guide (**Only for Vendor is Nvidia**)
1.7	2023-11-08	Roy Kuo	-	Supervyse OPF IPMI OEM Commnad Customization Guide (**Support all Vendor**)
1.8	2024-03-07	Jeff Chen	-	update "Update FRU file with SupervyseOPF IDE", and remove SkinnyJoe and Legocg1 in Supervyse OPF FRU Customization
1.9	2024-05-30	Wesley Wang	HungFu Lee	Add Channel config IDE configuration
2.0	2024-05-31	Jeff Chen	-	To merge OPF customization verification guide
2.1	2024-06-05	HungFu Lee	-	Add IDE configuration section to FRU customization

Revision Number	Date	Author	Reviewer	Summary
2.2	2024-07-08	HungFu Lee	-	revert IDE configuration section from FRU customization
2.3	2024-09-20	Roy Kuo	Brad Chang	Correction Supervyse OPF Device ID Customization
2.4	2024-10-18	Jeff Chen		Add PEF Porting Guide
2.5	2025-01-23	Jeff Chen		Chapter Review: FRU Configuration, PEF & Alert Configuration
2.6	2025-02-03	Will Xiao		Remove section: 2 Lan parameters customization Guide. This section is placed in porting guide
2.7	2025-02-26	Jeff Chen	Brad Chang	G4 platform has removed the USE_DEFAULT_FRU_FILE_default variable

1. Channel Table Configuration

1.1. Introduction



As the above picture depicts, "channels" are where the BMC uses to interact with other computing units. The BMC and Host are placed on a platform. Operator can use BMC through LAN to manage the platform, where BIOS and OS run on CPU, so-called Host. Host users, on the other hand, can use BMC through System Interface to inspect platform circumstances. System Interface nowadays is typically of three types: KCS, SSIF, BT.

In OPF, channels are configured using **channel table**, where users can configure channel attributes such as channel number, medium type, protocol type, I2C bus number/slave address etc. Channel table is part of the platform-configuration file named ***config.xml***; user can find this file at each platform on paths listed below.

1. For venders other than nvidia,

```
$(OPF_SDK)/meta-insyde/meta-$(VENDOR)/meta-$(PLATFORM)/recipes-
platform/ipmidcfg/files/plat/$(SKU)/config.xml
```

	archercity	*bhs*	*sp5*	*evb-ast2600*
intel	*default*	*default* *beechnut_city* *avenue_city*		

amd			*default* *plat0000-sku0000* *plat0000-sku0001* *plat0000-sku0002* *plat0000-sku0003* *plat0001-sku0000* *plat0001-sku0001* *plat0001-sku0002* *plat0001-sku0003*	
evb				*default*

2. For nvidia being the vendor,

G3 platforms:

```
$(OPF_SDK)/meta-insyde/meta-$(VENDOR)/meta-$(CPU_GPU)/recipes-
platform/ipmidcfg/files/plat-$(PLATFORM)/default/config.xml
```

G4 platforms:

```
$(OPF_SDK)/meta-insyde/meta-$(VENDOR)/meta-$(CPU_GPU)/recipes-
platform/ipmidcfg/files/plat-general/$(SKU)/config.xml
```

	G3 platform: *gh*	G4 platform: *gb200nvl*
nvidia	*Falcon* *SkinnyJoe* *NV-Starship* *Legoc2* *Legocg*1	*default* *gb200nvl-bmc-insyde*

1.2. Design and Restriction

As stated above, the channel table is embedded in config.xml. Channel attributes are programmed as XML tags.

At OPF build time, config.xml would be used to generate a file, named "***channel_config.json***", which is installed onto OPF image and loadable by OPF processes as resulted channel configuration. Users are thus required just to modify the config.xml within OPF SDK before they build the image if they want to customize their own channel configuration.

Channels are numbered from 0 to 15:
- channel 0 is fixed for **Primary IPMB**
- channel 1 is fixed for **LAN** eth0
- channel 7 is reserved for **SSIF** channel
- channel 15 is reserved for **KCS** and **BT** channel
- channel 12, 13, 14 are reserved and not to be used

other channels must be defined in channel table, either as user's wish or even if not used.

1.3. Configuration

1.3.1. config.xml

The following shows an example of channel table contained in a typical config.xml from an Intel bhs platform.

```
    <channel_table>
        <!-- Channel  0: IPMB, Primary IPMB -->
        <channel_entry
            u8ChannelNumber="0"
            u8Attendance="TRUE"
            u8InterfaceType="INTF_IPMB"
            u8InterfaceName="IPMB_PIPMB"
            u8I2CBusNumber="6"
            u8I2CSlaveAddress="0x20"
            opfChannelName="Ipmb"
            opfIsValid="true"
            opfChannelActiveSessions="0"
            opfChannelMediumType="ipmb"
            opfChannelProtocolType="ipmb-1.0"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true"/>
        <!-- Channel  1: LAN1 -->
        <channel_entry
            u8ChannelNumber="1"
            u8Attendance="TRUE"
            u8InterfaceType="INTF_LAN"
            u8InterfaceName="eth0"
            opfChannelName="eth0"
            opfIsValid="true"
            opfChannelActiveSessions="0"
            opfChannelMediumType="lan-802.3"
            opfChannelProtocolType="ipmb-1.0"
            opfChannelSessionSupported="multi-session"
            opfChannelIsIpmi="true"/>
        <!-- Channel  2: LAN2 -->
        <channel_entry
            u8ChannelNumber="2"
            u8Attendance="TRUE"
            u8InterfaceType="INTF_LAN"
            u8InterfaceName="eth1"
            opfChannelName="eth1"
            opfIsValid="true"
            opfChannelActiveSessions="0"
            opfChannelMediumType="lan-802.3"
            opfChannelProtocolType="ipmb-1.0"
            opfChannelSessionSupported="multi-session"
            opfChannelIsIpmi="true"/>
       <!-- Channel  3: Host Interface -->
        <channel_entry
            u8ChannelNumber="3"
            u8Attendance="TRUE"
            u8InterfaceType="INTF_LAN"
            u8InterfaceName="hostusb0"
            opfChannelName="hostusb0"
            opfIsValid="true"
            opfChannelActiveSessions="0"
            opfChannelMediumType="other-lan"
            opfChannelProtocolType="ipmb-1.0"
            opfChannelSessionSupported="multi-session"
```

```
            opfChannelIsIpmi="true" />
        <!-- Channel  4: Not used -->
        <channel_entry
            u8ChannelNumber="4"
            u8Attendance="FALSE"
            opfChannelName="RESERVED"
            opfIsValid="false"
            opfChannelActiveSessions="0"
            opfChannelMediumType="reserved"
            opfChannelProtocolType="na"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
        <!-- Channel  5: Not used -->
        <channel_entry
            u8ChannelNumber="5"
            u8Attendance="FALSE"
            opfChannelName="RESERVED"
            opfIsValid="false"
            opfChannelActiveSessions="0"
            opfChannelMediumType="reserved"
            opfChannelProtocolType="na"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
        <!-- Channel  6: IPMB, SMLINK0 -->
        <channel_entry
            u8ChannelNumber="6"
            u8Attendance="TRUE"
            u8InterfaceType="INTF_IPMB"
            u8InterfaceName="IPMB_SMLINK0"
            u8I2CBusNumber="5"
            u8I2CSlaveAddress="0x20"
            opfIsValid="true"
            opfChannelName="SMLINK"
            opfChannelActiveSessions="0"
            opfChannelMediumType="ipmb"
            opfChannelProtocolType="ipmb-1.0"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
        <!-- Channel  7: SSIF OPF:Not used-->
        <channel_entry
            u8ChannelNumber="7"
            u8Attendance="FALSE"
            u8I2CSlaveAddress="0x20"
            opfChannelName="RESERVED"
            opfIsValid="false"
            opfChannelActiveSessions="0"
            opfChannelMediumType="reserved"
            opfChannelProtocolType="na"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
        <!-- Channel  8: INTRABMC -->
        <channel_entry
            u8ChannelNumber="8"
            u8Attendance="TRUE"
            u8InterfaceType="INTF_INTRABMC"
            opfChannelName="INTRABMC"
            opfIsValid="true"
            opfChannelActiveSessions="0"
            opfChannelMediumType="oem"
            opfChannelProtocolType="oem"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
```

```
        <!-- Channel  9: IPMB, Secondary IPMB -->
        <channel_entry
            u8ChannelNumber="9"
            u8Attendance="TRUE"
            u8InterfaceType="INTF_IPMB"
            u8InterfaceName="IPMB_SIPMB"
            u8I2CBusNumber="9"
            u8I2CSlaveAddress="0x20"
            opfChannelName="SIPMB"
            opfIsValid="true"
            opfChannelActiveSessions="0"
            opfChannelMediumType="ipmb"
            opfChannelProtocolType="ipmb-1.0"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
        <!-- Channel  10: KCS, SMM -->
        <channel_entry
            u8ChannelNumber="10"
            u8Attendance="TRUE"
            u8InterfaceType="INTF_SMM"
            u8InterfaceName="kcs3"
            u16KCSIOBaseAddress="0x0ca4"
            opfIsValid="true"
            opfChannelName="ipmi_kcs4"
            opfChannelActiveSessions="0"
            opfChannelMediumType="system-interface"
            opfChannelProtocolType="kcs"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
        <!-- Channel 11: Serial OPF:Not used-->
        <channel_entry
            u8ChannelNumber="11"
            u8Attendance="FALSE"
            opfChannelName="RESERVED"
            opfIsValid="false"
            opfChannelActiveSessions="0"
            opfChannelMediumType="reserved"
            opfChannelProtocolType="na"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
        <!-- Channel 12: Reserved. Don't use. -->
        <channel_entry
            u8ChannelNumber="12"
            u8Attendance="FALSE"
            opfChannelName="RESERVED"
            opfIsValid="false"
            opfChannelActiveSessions="0"
            opfChannelMediumType="reserved"
            opfChannelProtocolType="na"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
      <!-- Channel 13: Reserved. Don't use. -->
        <channel_entry
            u8ChannelNumber="13"
            u8Attendance="FALSE"
            opfIsValid="false"
            opfChannelName="RESERVED"
            opfChannelActiveSessions="0"
            opfChannelMediumType="reserved"
            opfChannelProtocolType="na"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
```

```
        <!-- Channel 14: SELF. Don't use. -->
        <channel_entry
            u8ChannelNumber="14"
            u8Attendance="FALSE"
            opfIsValid="false"
            opfChannelName="SELF"
            opfChannelActiveSessions="0"
            opfChannelMediumType="unknown"
            opfChannelProtocolType="na"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
        <!-- Channel 15: KCS, SMS -->
        <channel_entry
            u8ChannelNumber="15"
            u8Attendance="TRUE"
            u8InterfaceType="INTF_KCS"
            u8InterfaceName="kcs2"
            u16KCSIOBaseAddress="0x0ca2"
            opfIsValid="true"
            opfChannelName="ipmi_kcs3"
            opfChannelActiveSessions="0"
            opfChannelMediumType="system-interface"
            opfChannelProtocolType="kcs"
            opfChannelSessionSupported="session-less"
            opfChannelIsIpmi="true" />
    </channel_table>
```

1.3.1.1. config.xml tag description

Attribute name	**Description**	**Possible value**	**Resulted field** in channel_config.json
u8ChannelNumber	Channel enumeration. Each number for one channel only.	0 - 15	[The index field]
u8Attendance	This channel used or not.	TRUE/FALSE	[deprecated]
u8InterfaceType	type describing interface this channel provides.	INTF_SMM INTF_IPMB INTF_INTRABMC INTF_LAN INTF_KCS/INTF_BT	[deprecated]
u8InterfaceName	name describing interface this channel provides.	user-defined	[deprecated]
u16KCSIOBaseAddress	mmap-ed base address for kcs channel	user-defined	[deprecated]

u8I2CBusNumber	I2C Bus Number for ipmb/smbus channel	user-defined	[deprecated]
u8I2CSlaveAddress	I2C slave address for ipmb/smbus channel	user-defined	[deprecated]
opfIsValid	whether this channel enabled to access IPMI command or not	true/false	*is_valid*
opfChannelName	name of this channel	user-defined	*name*
opfChannelActiveSessions	(reserved)	0	*active_sessions*
opfChannelMediumType	The information type used to transfer data on this channel	channel Medium Type: IPMI 2.0 spec. Table 6-3, *Channel Medium Type Numbers*	*medium_type*
opfChannelProtocolType	The type of low-level data exchanging protocol this channel uses	channel Protocol Type: IPMI 2.0 spec. Table 6-2, *Channel Protocol Type Numbers*	*protocol_type*
opfChannelSessionSupported	required to provide userId/privilege or not	session-less/ multi-session	session_supported
opfChannelIsIpmi	(reserved)	true	is_ipmi

1.3.1.2. Interface specific information (Legacy) (Only apply to codes existing before G3)

1.3.1.2.1. IPMB Interface

u8InterfaceType: Fixed to *INTF_IPMB*.
u8InterfaceName: *IPMB_PIPMB*, *IPMB_SIPMB*, *IPMB_SMLINK0*, *IPMB_PCIE*.
u8I2CBusNumber: user configurable.
u8I2CSlaveAddress: user configurable.

1.3.1.2.2. KCS Interface

u8InterfaceType: Fixed to *INTF_KCS*.
u8InterfaceName: user configurable.
"kcs0" - "kcs3" (points to "/dev/kcs0" - "/dev/kcs3")
u16KCSIOBaseAddress: user configurable.

1.3.1.2.3. LAN Interface

u8InterfaceType: Fixed to *INTF_LAN*.
u8InterfaceName: user configurable.
eth0, eth1 and usb0 (usb-over-lan).

1.3.1.2.4. SMM Interface

u8ChannelNumber: *user configurable*.
u8InterfaceType: Fixed to *INTF_SMM*.
u8InterfaceName: user configurable.
"kcs0" - "kcs3" (points to "/dev/kcs0" - "/dev/kcs3")
u16KCSIOBaseAddress - user configurable.
[SSIF channel]
u8I2CBusNumber - user configurable.
u8I2CSlaveAddress - user configurable.

1.3.2. How to use channel table on G3/G4 platform

1. Let's say the user has prepared a new channel, and would like to integrate it into BMC firmware

2. Firstly, the user figures out how to describe this new channel using attributes introduced in the previous section, both completely and consistently

3. Secondly, fill the entry residing in the channel table within ***config.xml***

4. To make OPF processes capable of recognizing when information is transferred upon this channel, the side other than BMC of this channel is required to carry the index of entry this channel occupies in the channel table within the payload transferred.

5. Finally, the handling side on BMC verifies the index carried in against that of the very entry in the ***channel_config.json*** whose attributes' value correctly describe this channel.

1.3.3. channel_config.json file content generated from the above example

The following shows the very channel_config.json generated from the above config.xml on an Intel BHS platform.

```
{
  "0" : {
    "name" : "Ipmb",
    "is_valid" : true,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "ipmb",
      "protocol_type" : "ipmb-1.0",
      "session_supported" : "session-less",
```

```
      "is_ipmi" : true
    }
  },
  "1" : {
    "name" : "eth0",
    "is_valid" : true,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "lan-802.3",
      "protocol_type" : "ipmb-1.0",
      "session_supported" : "multi-session",
      "is_ipmi" : true
    }
  },
  "2" : {
    "name" : "eth1",
    "is_valid" : true,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "lan-802.3",
      "protocol_type" : "ipmb-1.0",
      "session_supported" : "multi-session",
      "is_ipmi" : true
    }
  },
  "3" : {
    "name" : "hostusb0",
    "is_valid" : true,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "other-lan",
      "protocol_type" : "ipmb-1.0",
      "session_supported" : "multi-session",
      "is_ipmi" : true
    }
  },
  "4" : {
    "name" : "RESERVED",
    "is_valid" : false,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "reserved",
      "protocol_type" : "na",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "5" : {
    "name" : "RESERVED",
    "is_valid" : false,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "reserved",
      "protocol_type" : "na",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "6" : {
    "name" : "SMLINK",
    "is_valid" : true,
    "active_sessions" : 0,
```

```
    "channel_info" : {
      "medium_type" : "ipmb",
      "protocol_type" : "ipmb-1.0",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "7" : {
    "name" : "RESERVED",
    "is_valid" : false,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "reserved",
      "protocol_type" : "na",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "8" : {
    "name" : "INTRABMC",
    "is_valid" : true,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "oem",
      "protocol_type" : "oem",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "9" : {
    "name" : "SIPMB",
    "is_valid" : true,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "ipmb",
      "protocol_type" : "ipmb-1.0",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "10" : {
    "name" : "ipmi_kcs4",
    "is_valid" : true,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "system-interface",
      "protocol_type" : "kcs",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "11" : {
    "name" : "RESERVED",
    "is_valid" : false,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "reserved",
      "protocol_type" : "na",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
```

```
  "12" : {
    "name" : "RESERVED",
    "is_valid" : false,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "reserved",
      "protocol_type" : "na",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "13" : {
    "name" : "RESERVED",
    "is_valid" : false,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "reserved",
      "protocol_type" : "na",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "14" : {
    "name" : "SELF",
    "is_valid" : false,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "unknown",
      "protocol_type" : "na",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  },
  "15" : {
    "name" : "ipmi_kcs3",
    "is_valid" : true,
    "active_sessions" : 0,
    "channel_info" : {
      "medium_type" : "system-interface",
      "protocol_type" : "kcs",
      "session_supported" : "session-less",
      "is_ipmi" : true
    }
  }
}
```

1.3.4. channel_access.json

This is the default configuration of channel access. **Channel access applies only to session-ed channels**, i.e. LAN channel, SoL, USB-over-LAN etc.
The user can find and modify this file within SDK under the path listed below to provide the initial channel access configuration for OPF processes.

G3 platforms:

```
$(OPF_SDK)/meta-openbmc-mods/meta-common/recipes-phosphor/ipmi/phosphor-ipmi-
config/channel_access.json
```

G4 platforms:

Platform	channel_access.json path
Intel BHS	`$(OPF_SDK)/openbmc-meta-intel/meta-common/recipes-phosphor/ipmi/phosphor-ipmi-config/channel_access.json`
Nvidia gb200nvl	`$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gb200nvl/recipe-platform/ipmi/phosphor-ipmi-config/channel_access.json`

1.3.4.1. channel_access.json example

```
{
  "1" : {
    "access_mode" : "always_available",
    "user_auth_disabled" : false,
    "per_msg_auth_disabled" : false,
    "alerting_disabled" : false,
    "priv_limit" : "priv-admin"
  },
  "2" : {
    "access_mode" : "always_available",
    "user_auth_disabled" : false,
    "per_msg_auth_disabled" : false,
    "alerting_disabled" : false,
    "priv_limit" : "priv-admin"
  },
  "3" : {
    "access_mode" : "always_available",
    "user_auth_disabled" : false,
    "per_msg_auth_disabled" : false,
    "alerting_disabled" : false,
    "priv_limit" : "priv-admin"
  }
}
```

1.3.4.2. Tag explanation

Tag	Description	Possible Value
[Index field]	1-to-1 mapping to the index field in *channel_config.json*	0~15
access_mode	*IPMI 2.0 spec. Table 6-, Channel Access Modes*	disabled pre-boot always_available shared
user_auth_disabled	[Reserved] Not applied yet	false
per_msg_auth_disabled	[Reserved] Not applied yet	false
alerting_disabled	[Reserved] Not applied yet	false

priv_limit	Least privilege value necessary for users to hold in order to communicate upon this channel	priv-reserved priv-callback *priv-user* *priv-operator* *priv-admin* priv-oem

2. In-band Channel Service Porting

2.1. BT Channel

2.1.1. Introduction

- The Block Transfer (BT) interface is one of the supported BMC-to-SMS system interfaces.
- The BT Interface is so named because an entire block of message data is buffered before the management controller is notified of available data.

2.1.2. Configuration

2.1.2.1. BIOS

- Since BIOS often uses IPMI system channel to communicate with BMC, BIOS should enable BT channel support.

2.1.2.2. kernel

- Linux kernel config

```
CONFIG_ASPEED_BT_IPMI_BMC=y
```

Enable BT BMC driver.

- Linux kernel device tree

```
&ibt {
    ...
    status = "okay";
    ...
};
```

Source path: $(OPF_SDK)/meta-insyde/meta-$(VENDOR)/meta-$(PLATFORM)/ recipes-platform/linux-dts/linux-aspeed/insyde_overlay/arch/arm/boot/dts/XXX.dtsi

NOTE: this configuration is platform dependent.

2.1.2.3. platform

2.1.2.3.1. system channel service

- OPF assigns bitbake-variable **PREFERRED_PROVIDER_virtual/obmc-host-ipmi-hw** to determine which channel service is used as system channel. Available choices are:

1. "phosphor-ipmi-kcs"
2. "phosphor-ipmi-ssif"
3. **"phosphor-ipmi-bt"**

```
PREFERRED_PROVIDER_virtual/obmc-host-ipmi-hw = "phosphor-ipmi-bt"
```

- Customer can put this assignment into the machine configuration file,

Archercity:
$(SDK)/meta-insyde/meta-intel/meta-archercity/conf/machine/intel-ast2600-insyde.conf

Genoa:

$(SDK)/meta-insyde/meta-amd/meta-sp5/conf/machine/sp5-insyde.conf

BHS:

$(SDK)/meta-insyde/meta-intel/meta-bhs/conf/machine/intel-bhs-insyde.conf

2.1.2.3.2. IPMI channel table

- OPF uses **channel_table** tags in config.xml to assign the information of each IPMI channel.
- System channel should be fixed at channel 15.

Archercity:

$(SDK)/meta-insyde/meta-intel/meta-archercity/recipes-platform/ipmidcfg/files/plat/default/config.xml

Genoa:

$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/default/config.xml

$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/plat0000-sku0000/config.xml

$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/plat0000-sku0001/config.xml

$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/plat0000-sku0002/config.xml

$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/plat0000-sku0003/config.xml

$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/plat0001-sku0000/config.xml

$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/plat0001-sku0001/config.xml

$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/plat0001-sku0002/config.xml

$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/plat0001-sku0003/config.xml

BHS:

$(SDK)/meta-insyde/meta-intel/meta-bhs/recipes-platform/ipmidcfg/files/plat/default/config.xml

$(SDK)/meta-insyde/meta-intel/meta-bhs/recipes-platform/ipmidcfg/files/plat/avenue_city/config.xml

$(SDK)/meta-insyde/meta-intel/meta-bhs/recipes-platform/ipmidcfg/files/plat/beechnut_city/config.xml

```
<channel_table>
    <channel_entry
        u8ChannelNumber="15"
        u8Attendance="TRUE"
        u8InterfaceType="INTF_BT"
        opfIsValid="true"
        opfChannelName="ipmi_bt"
        opfChannelActiveSessions="0"
        opfChannelMediumType="system-interface"
        opfChannelProtocolType="bt-15"
        opfChannelSessionSupported="session-less"
        opfChannelIsIpmi="true" />
</channel_table>
```

u8ChannelNumber: **15** is recommended
u8InterfaceType: fixed **INTF_BT** for BT channel

opfChannelMediumType: fixed **system-interface**
opfChannelProtocolType: fixed **bt-15** for IPMI SPEC 1.5 and later implementation
opfChannelSessionSupported: fixed **session-less** for BT channel implementation;

2.2. SSIF Channel

2.2.1. Introduction

- The SMBus System Interface (SSIF) defines a SMBus-based system interface to the BMC.

2.2.2. Configuration

2.2.2.1. BIOS

- Since BIOS often uses IPMI system channel to communicate with BMC, BIOS should enable SSIF channel support.

2.2.2.2. kernel

- Linux kernel config

```
CONFIG_SSIF_IPMI_BMC=y
```

Enable SSIF BMC driver.

- Linux kernel device tree

```
&i2c0 {
    status = "okay";
    ssif-bmc@10 {
        compatible = "ssif-bmc";
        reg = <0x10>;
    };
};
```

NOTE: this configuration is platform dependent.

2.2.2.3. platform

2.2.2.3.1. system channel service

- OPF assigns bitbake variable **PREFERRED_PROVIDER_virtual/obmc-host-ipmi-hw** to determine which channel service is used for system interface. Available choices are:

1. phosphor-ipmi-kcs
2. **phosphor-ipmi-ssif**

```
PREFERRED_PROVIDER_virtual/obmc-host-ipmi-hw = "phosphor-ipmi-ssif"
```

3. phosphor-ipmi-bt

- Customer can put this assignment into machine configuration file,

Nvidia:

1.0

$(SDK)/meta-insyde/meta-nvidia/meta-gh/conf/machine/falcon-insyde.conf

$(SDK)/meta-insyde/meta-nvidia/meta-gh/conf/machine/starship-insyde.conf

$(SDK)/meta-insyde/meta-nvidia/meta-gh/conf/machine/skinnyjoe-insyde.conf

$(SDK)/meta-insyde/meta-nvidia/meta-gh/conf/machine/legoc2-insyde.conf

$(SDK)/meta-insyde/meta-nvidia/meta-gh/conf/machine/legocg1-insyde.conf

2.0

$(SDK)/meta-insyde/meta-nvidia/meta-gb200nvl/conf/machine/gb200nvl-bmc-insyde.conf

2.2.2.3.2. IPMI channel table

Nvidia:

1.0

$(SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-Falcon/default/config.xml

$(SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-NV-Starship/default/config.xml

$(SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-SkinnyJoe/default/config.xml

$(SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-Legoc2/default/config.xml

$(SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-Legocg1/default/config.xml

2.0

$(SDK)/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/ipmidcfg/files/plat-general/gb200nvl-bmc-insyde/config.xml

$(SDK)/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/ipmidcfg/files/plat-general/default/config.xml

```
<channel_table>
    <channel_entry
        u8ChannelNumber="7"
        u8Attendance="TRUE"
        u8InterfaceType="INTF_SSIF"
        u8I2CBusNumber="9"
        u8I2CSlaveAddress="0x10"
        opfIsValid="true"
        opfChannelName="ipmi_ssif"
        opfChannelActiveSessions="0"
        opfChannelMediumType="smbus-v2.0"
        opfChannelProtocolType="ipmi-smbus"
        opfChannelSessionSupported="session-less"
        opfChannelIsIpmi="true" />
</channel_table>
```

Next phase Fix SSIF channel number also to 15.

2.3. IPMB Channel

2.3.1. Introduction

IPMB is a protocol stack specified to facilitate communication between the BMC and Satellite Management Controllers (SMC)

So far as to Supervyse OPF, BMC only communicates through IPMB with the Intel ME (management engine) on G3 EGS platform

2.3.2. Configuration

2.3.2.1. Platform

2.3.2.1.1. Channel table

Intel EGS G3 platforms:

```
$(OPF_SDK)/meta-openbmc-mods/meta-common/recipes-phosphor/ipmi/phosphor-ipmi-ipmb/ipmb-channels.json
```

Actual content:

```
{
  "channels": [
    {
      "type": "me",
      "slave-path": "/dev/ipmb-5",
      "bmc-addr": 32,
      "remote-addr": 44
    },
    {
      "type": "ipmb",
      "slave-path": "/dev/ipmb-13",
      "bmc-addr": 32,
      "remote-addr": 32
    },
    {
      "type": "slot-ipmb",
      "slave-path": "/dev/ipmb-6",
      "bmc-addr": 18,
      "remote-addr": 176
    }
  ]
}
```

ipmb-channels.json fields explanation:

Field name	Possible Value	Description
type	me ipmb slot-ipmb	**me**: Intel EGS ME **slot-ipmb**: IPMB device on add-in cards; at most one entry of this type **ipmb**: otherwise (if any).
slave-path	*/dev/ipmb-**	*: the i2c bus number where i2c devices of bmc and remote controller reside

bmc-addr	0~255	I2c slave address of BMC
remote-addr	0~255	I2C slave address of remote controller
devIndex	*0~63* when type is “me”, “ipmb”; *0* otherwise	**[optional]** When there are multiple entries with the same type, this field is necessary to identify particular device/host.

3. FRU Configuration

3.1. Introduction

FRU (Field Replaceable Unit) typically refers to hardware components that can be replaced, such as a motherboard, power supply, or fan. Generally, FRU data (e.g., manufacturer or product number) is stored in non-volatile memory, such as EEPROM.

FruDevice is a service registered on D-Bus that scans all devices on I2C buses. When the scanned data conforms to the IPMI FRU format(Platform Management FRU Information Storage Definition v1.0), it is exposed on D-Bus.

Refer to Figure 4-1. Among the results of the FruDevice scan, only the object path highlighted in the block conforms to the IPMI FRU format. Command:

```
busctl tree xyz.openbmc_project.FruDevice
```

```
root@bmc-mac0007e93465e2:~# busctl tree xyz.openbmc_project.FruDevice
`-/xyz
  `-/xyz/openbmc_project
    `-/xyz/openbmc_project/FruDevice
      |-/xyz/openbmc_project/FruDevice/10_81
      |-/xyz/openbmc_project/FruDevice/12_80
      |-/xyz/openbmc_project/FruDevice/12_87
      |-/xyz/openbmc_project/FruDevice/15_47
      |-/xyz/openbmc_project/FruDevice/15_80
      |-/xyz/openbmc_project/FruDevice/15_81
      |-/xyz/openbmc_project/FruDevice/15_82
      |-/xyz/openbmc_project/FruDevice/15_83
      |-/xyz/openbmc_project/FruDevice/4_101
      |-/xyz/openbmc_project/FruDevice/4_103
      |-/xyz/openbmc_project/FruDevice/4_56
      |-/xyz/openbmc_project/FruDevice/5_114
      |-/xyz/openbmc_project/FruDevice/5_116
      |-/xyz/openbmc_project/FruDevice/5_117
      |-/xyz/openbmc_project/FruDevice/5_118
      |-/xyz/openbmc_project/FruDevice/5_40
      |-/xyz/openbmc_project/FruDevice/5_49
      |-/xyz/openbmc_project/FruDevice/5_50
      |-/xyz/openbmc_project/FruDevice/5_51
      |-/xyz/openbmc_project/FruDevice/5_53
      |-/xyz/openbmc_project/FruDevice/5_54
      |-/xyz/openbmc_project/FruDevice/5_55
      |-/xyz/openbmc_project/FruDevice/5_57
      |-/xyz/openbmc_project/FruDevice/5_58
      |-/xyz/openbmc_project/FruDevice/5_96
      |-/xyz/openbmc_project/FruDevice/5_97
      |-/xyz/openbmc_project/FruDevice/5_98
      |-/xyz/openbmc_project/FruDevice/5_99
      |-/xyz/openbmc_project/FruDevice/7_80
      |-/xyz/openbmc_project/FruDevice/8_80
      |-/xyz/openbmc_project/FruDevice/8_81
      |-/xyz/openbmc_project/FruDevice/9_92
      |-/xyz/openbmc_project/FruDevice/BHSL1U1SX16RSR
      |-/xyz/openbmc_project/FruDevice/BeechnutCity
      |-/xyz/openbmc_project/FruDevice/DC_SCM_2_0
      `-/xyz/openbmc_project/FruDevice/IS162F22
```

Figure 4-1. FruDevice scan result example

As shown in Figure 4-2, users can view the parsed FRU data from FruDevice, including the bus address:

```
busctl introspect xyz.openbmc_project.FruDevice <dbus object path>
root@bmc-mac0007e93465e2:~# busctl introspect xyz.openbmc_project.FruDevice /xyz/openbmc_project/FruDevice/BeechnutCity_0
NAME                                TYPE      SIGNATURE RESULT/VALUE            FLAGS
org.freedesktop.DBus.Introspectable interface -         -                       -
.Introspect                         method    -         s                       -
org.freedesktop.DBus.Peer           interface -         -                       -
.GetMachineId                       method    -         s                       -
.Ping                               method    -         -                       -
org.freedesktop.DBus.Properties     interface -         -                       -
.Get                                method    ss        v                       -
.GetAll                             method    s         a{sv}                   -
.Set                                method    ssv       -                       -
.PropertiesChanged                  signal    sa{sv}as  -                       -
xyz.openbmc_project.FruDevice       interface -         -                       -
.ADDRESS                            property  u         80                      emits-change
.BOARD_FRU_VERSION_ID               property  s         "0.02"                  emits-change
.BOARD_INFO_AM4                     property  s         "CountryCode:NA"        emits-change
.BOARD_LANGUAGE_CODE                property  s         "0"                     emits-change
.BOARD_MANUFACTURER                 property  s         "Intel Corporation"     emits-change
.BOARD_MANUFACTURE_DATE             property  s         "2023-01-06 - 00:00:00" emits-change
.BOARD_PART_NUMBER                  property  s         "M61861-205"            emits-change
.BOARD_PRODUCT_NAME                 property  s         "BeechnutCity"          emits-change
.BOARD_SERIAL_NUMBER                property  s         "..........."           emits-change
.BUS                                property  u         15                      emits-change
.CHASSIS_PART_NUMBER                property  s         "............"          emits-change
.CHASSIS_SERIAL_NUMBER              property  s         "............"          emits-change
.CHASSIS_TYPE                       property  s         "23"                    emits-change
.Common_Format_Version              property  s         "1"                     emits-change
.MUX                                property  s         ""                      emits-change
.PRODUCT_ASSET_TAG                  property  s         "bmc-test"              emits-change writable
.PRODUCT_LANGUAGE_CODE              property  s         "0"                     emits-change
.PRODUCT_MANUFACTURER               property  s         "Intel Corporation"     emits-change
.PRODUCT_PART_NUMBER                property  s         "........."             emits-change
.PRODUCT_PRODUCT_NAME               property  s         "BeechnutCity"          emits-change
.PRODUCT_SERIAL_NUMBER              property  s         "............"          emits-change
.PRODUCT_VERSION                    property  s         "....................."  emits-change
root@bmc-mac0007e93465e2:~#
```

Figure 4-2. FruDevice - BeechnutCity Baseboard FRU dbus interface

3.2. Configuration

The following is FRU configuration in this chapter:

- How to view and manage FRU data in EEPROM
  - Update FRU Data in EEPROM
- Baseboard FRU (ID 0) configuration
  - FRU data source: EEPROM or regular file
  - Predefined FRU file
- How to add a new FRU ID to FRU Table
  - Add the mapping between FRU ID and I2Cbus-address
  - Add FRU SDR

3.2.1. How to view and manage FRU data in EEPROM

As mentioned in the introduction, the FRU data stored in the EEPROM must conform to the IPMI FRU format before it is exposed on D-Bus.

Typically, the FRU manufacturer writes the default data into the EEPROM. Users can verify it using the bus address and the following command:

For one-byte address EEPROM:

```
i2cdump -f -y <bus> <address>
```

Example for G4 BeechnutCity platform Baseboard FRU (ID 0):

```
root@bmc-mac0007e93465e2:~# i2cdump -f -y 15 80
No size specified (using byte-data access)
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 01 00 01 05 10 00 00 e9 01 04 17 cc 2e 2e 2e 2e    ?.???..?????....
10: 2e 2e 2e 2e 2e 2e 2e 2e cc 2e 2e 2e 2e 2e 2e 2e    ........?.......
20: 2e 2e 2e 2e 2e c1 00 3b 01 0b 00 e0 cd d8 d1 49    .....?.;??.????I
30: 6e 74 65 6c 20 43 6f 72 70 6f 72 61 74 69 6f 6e    ntel Corporation
40: cc 42 65 65 63 68 6e 75 74 43 69 74 79 cc 2e 2e    ?BeechnutCity?..
50: 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e ca 4d 36 31 38 36    ..........?M6186
60: 31 2d 32 30 35 c4 30 2e 30 32 c0 c0 c0 ce 43 6f    1-205?0.02????Co
70: 75 6e 74 72 79 43 6f 64 65 3a 4e 41 c1 00 00 ff    untryCode:NA?...
80: 01 0c 00 d1 49 6e 74 65 6c 20 43 6f 72 70 6f 72    ??.?Intel Corpor
90: 61 74 69 6f 6e cc 42 65 65 63 68 6e 75 74 43 69    ation?BeechnutCi
a0: 74 79 ca 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e d4 2e 2e    ty?..........?..
b0: 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e    ................
c0: 2e 2e cc 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e c8    ..?............?
d0: 62 6d 63 2d 74 65 73 74 c0 c1 00 00 00 00 00 85    bmc-test??.....?
e0: 01 02 00 10 01 00 10 ff 00 00 00 00 00 00 00 00    ??.??.?........
f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
root@bmc-mac0007e93465e2:~#
```

Figure 4-3. BeechnutCity Baseboard FRU data in one-byte address EEPROM

For two-bytes address EEPROM:

```
i2ctransfer -f -y <bus> w2@<address> <offset 1> <offset 2> r<read bytes>
```

- Offset specifies where to start reading, while read bytes indicates how many bytes to read.

Example for G3 Starship ProcMod_0 FRU (ID 5):

```
root@BMCXXXXXXXXXXXX:~# i2cdump -f -y 2 80
No size specified (using byte-data access)
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 00 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
10: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
20: ff 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
30: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
40: ff 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
50: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
60: ff 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
70: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
80: ff 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
90: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
a0: ff 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
b0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
c0: ff 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
d0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
e0: ff 00 ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
f0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
root@BMCXXXXXXXXXXXX:~# i2ctransfer -f -y 2 w2@0x50 0x0 0x0 r256
0x01 0x00 0x00 0x01 0x08 0x00 0x00 0xf6 0x01 0x07 0x19 0x30 0x44 0xdd 0xc6 0x4e 0x5
6 0x49 0x44 0x49 0x41 0xc5 0x50 0x47 0x35 0x33 0x35 0xcd 0x31 0x35 0x38 0x32 0x38 0
x32 0x33 0x37 0x32 0x30 0x30 0x31 0x32 0xd2 0x36 0x39 0x39 0x2d 0x32 0x47 0x35 0x33
 0x35 0x2d 0x30 0x32 0x30 0x30 0x2d 0x51 0x53 0x33 0xc0 0xc1 0x00 0x7d 0x01 0x09 0x
19 0xc6 0x4e 0x56 0x49 0x44 0x49 0x41 0xc2 0x43 0x32 0xd2 0x39 0x30 0x30 0x2d 0x32
0x47 0x35 0x33 0x35 0x2d 0x30 0x30 0x30 0x30 0x2d 0x30 0x30 0x30 0xc5 0x42 0x2d 0x5
2 0x30 0x30 0xcd 0x31 0x35 0x38 0x32 0x38 0x32 0x33 0x37 0x32 0x30 0x30 0x31 0x32 0
xc0 0xcc 0x46 0x52 0x55 0x5f 0x56 0x65 0x72 0x5f 0x30 0x2e 0x30 0x32 0xc1 0x00 0x00
 0x00 0x00 0x9c 0x30 0x32 0xc1 0x00 0x00 0x00 0x00 0x9c 0xff 0xff 0xff 0xff 0xff 0x
ff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xf
f 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0
xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0x
ff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
0xff 0xff 0xff 0xff 0xff 0xff 0xff
root@BMCXXXXXXXXXXXX:~#
```

Figure 4-4. Falcon platform ProcMod_0 FRU data in two-bytes address EEPROM

3.2.1.1. Update FRU Data in EEPROM

If the EEPROM content is empty or does not conform to the IPMI FRU format, the user can use the following command to update it:

Hint: Users should back up data before modifying FRU data to prevent data loss!

For one-byte address EEPROM:

```
i2ctransfer -f -y <bus> w<data num>@<address> <offset> <data to write>
```

For two-bytes address EEPROM:

```
i2ctransfer -f -y <bus> w<data num>@<address> <offset1> <offset2> <data to write>
```

Example for G4 BeechnutCity platform Baseboard FRU (ID 0):

```
root@bmc-mac0007e93465e2:~# i2cdump -f -y 15 0x50
No size specified (using byte-data access)
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 01 00 01 05 10 00 00 e9 01 04 17 cc 2e 2e 2e 2e    ?.???..?????....
10: 2e 2e 2e 2e 2e 2e 2e 2e cc 2e 2e 2e 2e 2e 2e 2e    ........?.......
20: 2e 2e 2e 2e 2e c1 00 3b 01 0b 00 e0 cd d8 d1 49    .....?.;??.????I
30: 6e 74 65 6c 20 43 6f 72 70 6f 72 61 74 69 6f 6e    ntel Corporation
40: cc 42 65 65 63 68 6e 75 74 43 69 74 79 cc 2e 2e    ?BeechnutCity?..
50: 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e ca 4d 36 31 38 36    ..........?M6186
60: 31 2d 32 30 35 c4 30 2e 30 32 c0 c0 c0 ce 43 6f    1-205?0.02????Co
70: 75 6e 74 72 79 43 6f 64 65 3a 4e 41 c1 00 00 ff    untryCode:NA?...
80: 01 0c 00 d1 49 6e 74 65 6c 20 43 6f 72 70 6f 72    ??.?Intel Corpor
90: 61 74 69 6f 6e cc 42 65 65 63 68 6e 75 74 43 69    ation?BeechnutCi
a0: 74 79 ca 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e d4 2e 2e    ty?..........?..
b0: 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e    ................
c0: 2e 2e cc 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e c8    ..?............?
d0: 62 6d 63 2d 74 65 73 74 c0 c1 00 00 00 00 00 85    bmc-test??.....?
e0: 01 02 00 10 01 00 10 ff 00 00 00 00 00 00 00 00    ??.??.?........
f0: 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00    ................
root@bmc-mac0007e93465e2:~# i2ctransfer -f -y 15 w9@0x50 0xf0 0xff 0xff 0xff 0xff 0xff 0xff 0xff 0xff
root@bmc-mac0007e93465e2:~# i2cdump -f -y 15 0x50
No size specified (using byte-data access)
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 01 00 01 05 10 00 00 e9 01 04 17 cc 2e 2e 2e 2e    ?.???..?????....
10: 2e 2e 2e 2e 2e 2e 2e 2e cc 2e 2e 2e 2e 2e 2e 2e    ........?.......
20: 2e 2e 2e 2e 2e c1 00 3b 01 0b 00 e0 cd d8 d1 49    .....?.;??.????I
30: 6e 74 65 6c 20 43 6f 72 70 6f 72 61 74 69 6f 6e    ntel Corporation
40: cc 42 65 65 63 68 6e 75 74 43 69 74 79 cc 2e 2e    ?BeechnutCity?..
50: 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e ca 4d 36 31 38 36    ..........?M6186
60: 31 2d 32 30 35 c4 30 2e 30 32 c0 c0 c0 ce 43 6f    1-205?0.02????Co
70: 75 6e 74 72 79 43 6f 64 65 3a 4e 41 c1 00 00 ff    untryCode:NA?...
80: 01 0c 00 d1 49 6e 74 65 6c 20 43 6f 72 70 6f 72    ??.?Intel Corpor
90: 61 74 69 6f 6e cc 42 65 65 63 68 6e 75 74 43 69    ation?BeechnutCi
a0: 74 79 ca 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e d4 2e 2e    ty?..........?..
b0: 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e    ................
c0: 2e 2e cc 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e 2e c8    ..?............?
d0: 62 6d 63 2d 74 65 73 74 c0 c1 00 00 00 00 00 85    bmc-test??.....?
e0: 01 02 00 10 01 00 10 ff 00 00 00 00 00 00 00 00    ??.??.?........
f0: ff ff ff ff ff ff ff ff 00 00 00 00 00 00 00 00    ................
```

Figure 4-5. Update EEPROM content with i2ctransfer command

If the FRU has been assigned a FRU ID and conforms to the IPMI FRU format, the user can directly use the ipmitool command to update the FRU data:

```
ipmitool fru write <FRU ID> <FRU binary>
```

Example for G3 Starship platform ProcMod_0 FRU (ID 5):

```
root@BMC000000000000:~# ipmitool fru print 5
 Board Mfg Date        : Sat Nov 18 01:23:00 2023 UTC
 Board Mfg             : foxconnnn
 Board Product         : PG535
 Board Serial          : AAAAAAAAASBBWWXXX
 Board Part Number     : 02030CV00-600-G
 Board Extra           : Version: 0A
 Board Area Checksum   : OK
 Product Manufacturer  : Foxconn
 Product Name          : Falcon 1U Node
 Product Part Number   : 1A62WCD00-600-G
 Product Version       : Ver_1.0
 Product Serial        : XXXXXXXXXXXXX
 Product Area Checksum : OK
root@BMC000000000000:~# ls
bmcweb_persistent_data.json  fru5.bin  test.bin
root@BMC000000000000:~#
root@BMC000000000000:~# ipmitool fru write 5 fru5.bin
Fru Size         : 256 bytes
Size to Write    : 256 bytes
root@BMC000000000000:~# ipmitool fru print 5
 Board Mfg Date        : Fri Jul  7 11:42:00 2023 UTC
 Board Mfg             : NVIDIA
 Board Product         : PG530
 Board Serial          : THAKN2248000S
 Board Part Number     : 699-2G530-0204-QS5
 Board Area Checksum   : OK
 Product Manufacturer  : NVIDIA
 Product Name          : GH200 120GB
 Product Part Number   : 900-2G530-0000-000
 Product Version       : A-R00
 Product Serial        : 1642023000517
 Product Area Checksum : OK
root@BMC000000000000:~#
```

Figure 4-6. Update EEPROM content with ipmitool command

3.2.2. Baseboard FRU (ID 0) configuration

Baseboard FRU (ID 0) data source can be switched between EEPROM and regular file.

Default baseboard FRU data source:

G3:

- Intel – ArcherCity CRB: from file
- AMD – Turin CRB: from EEPROM
- AMD - Onyx / Quartz CRB: from EEPROM
- Nvidia – Falcon CRB: from EEPROM
- Nvidia - NV-Starship CRB: from EEPROM

- Nvidia - MGX C2 CRB: from EEPROM
- Nvidia - MGX CG1 CRB: from EEPROM
- Ast2600evb: from file

G4:

- Intel – BeechnutCity / ArcherCity CRB: from EEPROM
- Nvidia - Oberon CRB: from EEPROM
- Ast2600evb: from file
- Ast2700evb: from file

3.2.2.1. FRU data source from EEPROM or regular file

The baseboard FRU data source setting is specified in the EEPROM_FRU variable in decodeBoardID.sh.

Note: When selecting regular file (EEPROM_FRU = false), if the predefined FRU file does not exist, The BMC will generate a basic baseboard FRU file through SetBaseboardFru.service at startup.

- decodeBoardID.sh path in **G3**:
  - For **Intel ArcherCity** CRB

```
${G3_SDK}/meta-insyde/meta-intel/meta-archercity/recipes-platform/fru/default-fru/decodeBoardID.sh
```

  - For Nvidia **Falcon** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/fru/default-fru/decodeBoardID-Falcon.sh
```

  - For Nvidia **Starship** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/fru/default-fru/decodeBoardID-NV-Starship.sh
```

  - For Nvidia **MGX C2** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/fru/default-fru/decodeBoardID-Legoc2.sh
```

  - For Nvidia **MGX CG1** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/fru/default-fru/decodeBoardID-Legocg1.sh
```

  - For **AMD Onyx / Quartz** CRBs

```
${G3_SDK}/meta-insyde/meta-amd/meta-sp5/recipes-platform/fru/default-fru/decodeBoardID.sh
```

  - For **AMD Turin** CRB

```
${G3_SDK}/meta-insyde/meta-amd/meta-turin/recipes-platform/fru/default-fru/decodeBoardID.sh
```

  - For **ast2600evb**

```
${G3_SDK}/meta-insyde/meta-evb/meta-evb-ast2600/recipes-platform/fru/default-fru/decodeBoardID.sh
```

- decodeBoardID.sh path in **G4**:
  - For Intel **BeechnutCity / AvenueCity** CRBs

```
${G4_SDK}/meta-insyde/meta-intel/meta-bhs/recipes-platform/fru/default-
fru/decodeBoardID.sh
```

- For Nvidia **Oberon** CRB

```
${G4_SDK}/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/fru/default-
fru/decodeBoardID.sh
```

- For **ast2600evb**

```
$(G4_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-phosphor/fru/default-
fru/decodeBoardID.sh
```

- For **ast2700evb**

```
$(G4_SDK)/meta-insyde/meta-evb/meta-ast2700evb/recipes-phosphor/fru/default-
fru/decodeBoardID.sh
```

- decodeBoardID.sh path of new platform:
  - For **new platform**

```
$(G4_SDK)/meta-base/meta-${new_platform}/recipes-platform/fru/default-
fru/decodeBoardID.sh
```

The following takes decodeBoardID.sh of AMD-Onyx platform as an example:

- When EEPROM_FRU = false, FruDevice will use regular file as the baseboard FRU data.
- When EEPROM_FRU = true, FruDevice will use EEPROM as the baseboard FRU data.

```
# this script uses the BOARD_ID set from checkFru.sh and provides the NAME,
# PRODID, and EEPROM_FRU values for this platform
decode_board_id() {
    # Read Board ID from u-boot env
    PRODID=$board_id
    EEPROM_FRU=true                                <<<<<<<<<<<<<<<<<<<<<<<<<<<<<
    case $board_id in
       ...
    esac
}
```

3.2.2.2. Predefined FRU file

When selecting a file as the baseboard FRU data source, if the user does not want to use the FRU file generated by SetBaseboardFru.service, the user can use the predefined FRU file.

3.2.2.2.1. Predefined FRU file path

Note, please make sure the file name is "baseboard.fru.bin".

G3:

- For **Intel ArcherCity** CRB

```
${G3_SDK}/meta-insyde/meta-intel/meta-archercity/recipes-
platform/ipmidcfg/files/plat/default/baseboard.fru.bin
```

- For Nvidia **Falcon** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-
Falcon/default/baseboard.fru.bin
```

- For Nvidia **Starship** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-NV-
Starship/default/baseboard.fru.bin
```

- For Nvidia **MGX C2** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-Legoc2/default/baseboard.fru.bin
```

- For Nvidia **MGX CG1** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-Legocg1/default/baseboard.fru.bin
```

- For **AMD Onyx / Quartz** CRBs

```
${G3_SDK}/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmidcfg/files/plat/default/baseboard.fru.bin
```

- For **AMD Turin** CRB

```
${G3_SDK}/meta-insyde/meta-amd/meta-turin/recipes-platform/ipmidcfg/files/plat/default/baseboard.fru.bin
```

- For **ast2600evb**

```
${G3_SDK}/meta-insyde/meta-evb/meta-evb-ast2600/recipes-platform/ipmidcfg/files/plat/default/baseboard.fru.bin
```

G4:

- For Intel **BeechnutCity** CRBs

```
${G4_SDK}/meta-insyde/meta-intel/meta-bhs/recipes-platform/ipmidcfg/files/plat/beechnut_city/baseboard.fru.bin
```

- For Intel **AvenueCity** CRBs

```
${G4_SDK}/meta-insyde/meta-intel/meta-bhs/recipes-platform/ipmidcfg/files/plat/avenue_city/baseboard.fru.bin
```

- For Nvidia **Oberon** CRB

```
${G4_SDK}/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/ipmidcfg/files/plat-general/gb200nvl-bmc-insyde/baseboard.fru.bin
```

- For **ast2600evb**

```
$(G4_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-platform/ipmidcfg/files/plat/ast2600-evb-insyde/baseboard.fru.bin
```

- For **ast2700evb**

```
$(G4_SDK)/meta-insyde/meta-evb/meta-ast2700evb/recipes-platform/ipmidcfg/files/plat/ast2700-evb-insyde/baseboard.fru.bin
```

New platform:

- For **new platform**

```
$(G4_SDK)/meta-base/meta-${new_platform}/recipes-platform/ipmidcfg/files/plat/${SKU}/baseboard.fru.bin
```

3.2.2.2.2. ipmidcfg recipe configuration

G3:

the user needs to modify ipmidcfg-native_%.bbappend in the ipmidcfg layer.

The following takes Intel ArcherCity platform as an example, path:

```
$(SDK)/meta-insyde/meta-intel/meta-archercity/recipes-platform/ipmidcfg/
```

- USE_DEFAULT_FRU_FILE_default='y' means use pre-defined FRU file.
- USE_DEFAULT_FRU_FILE_default='n' means not using the pre-defined FRU file.

```
FILESEXTRAPATHS:prepend := "${THISDIR}/files:"
USE_DEFAULT_FRU_FILE_default='n'                        <<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<

SRC_URI:append = "file://plat/default/AC-Baseboard-
Immutable.json;subdir=${IPMIDCFGFILES} \
                  file://plat/default/AC-Chassis-
Immutable.json;subdir=${IPMIDCFGFILES} \
                  file://plat/default/config.xml;subdir=${IPMIDCFGFILES} \
                  ${@bb.utils.contains('USE_DEFAULT_FRU_FILE_default', 'y', '
file://plat/default/baseboard.fru.bin;subdir=${IPMIDCFGFILES}', '', d)} \
```

G4:

The user does not need to configure the ipmidcfg recipe.

3.2.2.3. Verification

If the baseboard FRU data is from a regular file, whether predefined or generated, the user can find it at the following path:

```
/etc/fru/baseboard.fru.bin
```

Take G3 Intel ArcherCity platform as example, refer to Figure 4-7:

```
root@intel-obmc:~# ipmitool fru print 0
 Chassis Type          : Rack Mount Chassis
 Chassis Part Number   : ..................
 Chassis Serial        : ..................
 Chassis Extra         : ................................
 Chassis Extra         : ................................
 Board Mfg Date        : Unspecified
 Board Mfg             : Intel Corporation
 Board Product         : ArcherCity
 Board Serial          : ............
 Board Part Number     : ..........
 Product Manufacturer  : Intel Corporation
 Product Name          : ArcherCity
 Product Part Number   : ..........
 Product Version       : ....................
 Product Serial        : ............
 Product Asset Tag     : ....................
root@intel-obmc:~#
root@intel-obmc:~# cd /etc/fru
root@intel-obmc:/etc/fru# ls
baseboard.fru.bin
root@intel-obmc:/etc/fru# hexdump -C baseboard.fru.bin
00000000  01 01 02 10 1a 00 00 d2  01 00 00 00 00 00 00 01  |................|
00000010  01 0e 17 d2 2e 2e 2e 2e  2e 2e 2e 2e 2e 2e 2e 2e  |................|
00000020  2e 2e 2e 2e 2e 2e d2 2e  2e 2e 2e 2e 2e 2e 2e 2e  |................|
00000030  2e 2e 2e 2e 2e 2e 2e 2e  2e df 2e 2e 2e 2e 2e 2e  |................|
00000040  2e 2e 2e 2e 2e 2e 2e 2e  2e 2e 2e 2e 2e 2e 2e 2e  |................|
00000050  2e 2e 2e 2e 2e 2e 2e 2e  2e df 2e 2e 2e 2e 2e 2e  |................|
00000060  2e 2e 2e 2e 2e 2e 2e 2e  2e 2e 2e 2e 2e 2e 2e 2e  |................|
00000070  2e 2e 2e 2e 2e 2e 2e 2e  2e c1 00 00 00 00 00 1b  |................|
00000080  01 0a 00 00 00 00 d1 49  6e 74 65 6c 20 43 6f 72  |.......Intel Cor|
00000090  70 6f 72 61 74 69 6f 6e  ca 41 72 63 68 65 72 43  |poration.ArcherC|
000000a0  69 74 79 cc 2e 2e 2e 2e  2e 2e 2e 2e 2e 2e 2e 2e  |ity.............|
000000b0  ca 2e 2e 2e 2e 2e 2e 2e  2e 2e 2e cc 46 52 55 20  |............FRU |
000000c0  56 65 72 20 30 2e 30 31  c1 00 00 00 00 00 00 90  |Ver 0.01........|
000000d0  01 0d 00 d1 49 6e 74 65  6c 20 43 6f 72 70 6f 72  |....Intel Corpor|
000000e0  61 74 69 6f 6e ca 41 72  63 68 65 72 43 69 74 79  |ation.ArcherCity|
000000f0  ca 2e 2e 2e 2e 2e 2e 2e  2e 2e 2e d4 2e 2e 2e 2e  |................|
00000100  2e 2e 2e 2e 2e 2e 2e 2e  2e 2e 2e 2e 2e 2e 2e 2e  |................|
00000110  cc 2e 2e 2e 2e 2e 2e 2e  2e 2e 2e 2e 2e d4 2e 2e  |................|
00000120  2e 2e 2e 2e 2e 2e 2e 2e  2e 2e 2e 2e 2e 2e 2e 2e  |................|
00000130  2e 2e c0 c1 00 00 00 da  00 00 00 00 00 00 00 00  |................|
00000140  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00 00  |................|
*
000001f0  00 00 00 00 00 00 00 00  00 00 00 00 00 00 00     |...............|
000001ff
root@intel-obmc:/etc/fru#
```

Figure 4-7. Baseboard FRU data from regular file

And the bus and address on FruDevice will display 0, refer to Figure 4-8:

```
root@intel-obmc:~# busctl introspect xyz.openbmc_project.FruDevice /xyz/openbmc_project/FruDevice/ArcherCity
NAME                                TYPE      SIGNATURE RESULT/VALUE                     FLAGS
org.freedesktop.DBus.Introspectable interface -         -                                -
.Introspect                         method    -         s                                -
org.freedesktop.DBus.Peer           interface -         -                                -
.GetMachineId                       method    -         s                                -
.Ping                               method    -         -                                -
org.freedesktop.DBus.Properties     interface -         -                                -
.Get                                method    ss        v                                -
.GetAll                             method    s         a{sv}                            -
.Set                                method    ssv       -                                -
.PropertiesChanged                  signal    sa{sv}as  -                                -
xyz.openbmc_project.FruDevice       interface -         -                                -
.ADDRESS                            property  u         0                                emits-change
.BOARD_FRU_VERSION_ID               property  s         "FRU Ver 0.01"                   emits-change
.BOARD_LANGUAGE_CODE                property  s         "0"                              emits-change
.BOARD_MANUFACTURER                 property  s         "Intel Corporation"              emits-change
.BOARD_MANUFACTURE_DATE             property  s         "1996-01-01 - 00:00:00"          emits-change
.BOARD_PART_NUMBER                  property  s         "........."                      emits-change
.BOARD_PRODUCT_NAME                 property  s         "ArcherCity"                     emits-change
.BOARD_SERIAL_NUMBER                property  s         "..........."                    emits-change
.BUS                                property  u         0                                emits-change
.CHASSIS_INFO_AM1                   property  s         "............................." emits-change
.CHASSIS_INFO_AM2                   property  s         "............................." emits-change
.CHASSIS_PART_NUMBER                property  s         "................"               emits-change
.CHASSIS_SERIAL_NUMBER              property  s         "................"               emits-change
.CHASSIS_TYPE                       property  s         "23"                             emits-change
.Common_Format_Version              property  s         "1"                              emits-change
.PRODUCT_ASSET_TAG                  property  s         "...................."           emits-change writable
.PRODUCT_LANGUAGE_CODE              property  s         "0"                              emits-change
.PRODUCT_MANUFACTURER               property  s         "Intel Corporation"              emits-change
.PRODUCT_PART_NUMBER                property  s         "........."                      emits-change
.PRODUCT_PRODUCT_NAME               property  s         "ArcherCity"                     emits-change
.PRODUCT_SERIAL_NUMBER              property  s         "..........."                    emits-change
.PRODUCT_VERSION                    property  s         "...................."           emits-change
```

Figure 4-8. Bus and address of baseboard FRU data from regular file

If the baseboard FRU data is from EEPROM, the user can view the contents of the EEPROM using the i2ctool provided by Linux:

Take G3 NV-Falcon platform as example, refer to Figure 4-9:

```
root@nvidia-obmc:~# ipmitool fru print 0
 Board Mfg Date        : Mon Apr 22 20:00:00 2024 UTC
 Board Mfg             : NVIDIA
 Board Product         : P3809
 Board Serial          : 1581524930195
 Board Part Number     : 699-13809-0404-500
 Board Extra           : 01
 Board Extra           : MAC: 3C:6D:66:05:55:35
 Board Area Checksum   : OK
 Product Manufacturer  : NVIDIA
 Product Name          : P4975
 Product Part Number   : 670-24975-0000-000
 Product Version       : AE.1
 Product Serial        : 1581524930195
 Product Area Checksum : OK
root@nvidia-obmc:~#
root@nvidia-obmc:~# i2cdump -f -y 10 80
No size specified (using byte-data access)
     0  1  2  3  4  5  6  7  8  9  a  b  c  d  e  f    0123456789abcdef
00: 01 00 00 01 0b 00 00 f3 01 0a 19 90 31 e3 c6 4e    ?..??..?????1??N
10: 56 49 44 49 41 c5 50 33 38 30 39 cd 31 35 38 31    VIDIA?P3809?1581
20: 35 32 34 39 33 30 31 39 35 d2 36 39 39 2d 31 33    524930195?699-13
30: 38 30 39 2d 30 34 30 34 2d 35 30 30 c0 01 01 d6    809-0404-500????
40: 4d 41 43 3a 20 33 43 3a 36 44 3a 36 36 3a 30 35    MAC: 3C:6D:66:05
50: 3a 35 35 3a 33 35 c1 c0 01 08 19 c6 4e 56 49 44    :55:35??????NVID
60: 49 41 c5 50 34 39 37 35 d2 36 37 30 2d 32 34 39    IA?P4975?670-249
70: 37 35 2d 30 30 30 30 2d 30 30 30 c4 41 45 2e 31    75-0000-000?AE.1
80: cd 31 35 38 31 35 32 34 39 33 30 31 39 35 c0 c4    ?1581524930195??
90: 76 30 2e 31 c1 00 00 b9 ff ff ff ff ff ff ff ff    v0.1?..?........
a0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
b0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
c0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
d0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
e0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
f0: ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff ff    ................
root@nvidia-obmc:~#
```

Figure 4-9. Baseboard FRU data from EEPROM

The bus and address on FruDevice will indicate the exact location on I2C bus, shown as Figure 4-10:

```
root@nvidia-obmc:~# busctl introspect xyz.openbmc_project.FruDevice /xyz/openbmc_project/FruDevice/P3809
NAME                                TYPE      SIGNATURE RESULT/VALUE             FLAGS
org.freedesktop.DBus.Introspectable interface -         -                        -
.Introspect                         method    -         s                        -
org.freedesktop.DBus.Peer           interface -         -                        -
.GetMachineId                       method    -         s                        -
.Ping                               method    -         -                        -
org.freedesktop.DBus.Properties     interface -         -                        -
.Get                                method    ss        v                        -
.GetAll                             method    s         a{sv}                    -
.Set                                method    ssv       -                        -
.PropertiesChanged                  signal    sa{sv}as  -                        -
xyz.openbmc_project.FruDevice       interface -         -                        -
.ADDRESS                            property  u         80                       emits-change
.BOARD_INFO_AM1                     property  s         "01"                     emits-change
.BOARD_INFO_AM2                     property  s         "MAC: 3C:6D:66:05:55:35" emits-change
.BOARD_LANGUAGE_CODE                property  s         "25"                     emits-change
.BOARD_MANUFACTURER                 property  s         "NVIDIA"                 emits-change
.BOARD_MANUFACTURE_DATE             property  s         "2024-04-22T20:00:00Z"   emits-change
.BOARD_PART_NUMBER                  property  s         "699-13809-0404-500"     emits-change
.BOARD_PRODUCT_NAME                 property  s         "P3809"                  emits-change
.BOARD_SERIAL_NUMBER                property  s         "1581524930195"          emits-change
.BUS                                property  u         10                       emits-change
.Common_Format_Version              property  s         "1"                      emits-change
.PRODUCT_ASSET_TAG                  property  s         ""                       emits-change writable
.PRODUCT_FRU_VERSION_ID             property  s         "v0.1"                   emits-change
.PRODUCT_LANGUAGE_CODE              property  s         "25"                     emits-change
.PRODUCT_MANUFACTURER               property  s         "NVIDIA"                 emits-change
.PRODUCT_PART_NUMBER                property  s         "670-24975-0000-000"     emits-change
.PRODUCT_PRODUCT_NAME               property  s         "P4975"                  emits-change
.PRODUCT_SERIAL_NUMBER              property  s         "1581524930195"          emits-change
.PRODUCT_VERSION                    property  s         "AE.1"                   emits-change
root@nvidia-obmc:~#
```

Figure 4-10. Bus and address of baseboard FRU data from EEPROM

3.2.3. How to add a new FRU ID to FRU Table

For the FRU data that has been exposed on D-Bus, the user can configure this section to display it in the FRU table.

The following is the flow of "ipmitool fru print" command:

1. Find all FRU IDs from FRU SDR.
2. Use the FRU ID to find the mapping I2Cbus-address from the Entity Manager service. (FRU ID and bus-address must be one-to-one)

3. Use I2Cbus-address to obtain FRU Data from FruDevice service.

So, if the user wants to customize a new FRU ID, they need to perform the following configuration:

- Add the mapping between FRU ID and I2Cbus-address in Entity Manager service
- Add FRU SDR

Path to the entity-manager configuration file and recipe:
G3:

- For Intel **ArcherCity** CRB

```
$(SDK)/meta-insyde/meta-intel/meta-archercity/recipes-platform/
```

- For Nvidia **Falcon / Starship / MGX C2 / MGX CG1** CRBs

```
$(SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/
```

- For AMD **Turin** CRB

```
$(SDK)/meta-insyde/meta-amd/meta-turin/recipes-platform/
```

- For AMD **Onyx / Quartz** CRBs

```
$(SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/
```

- For **ast2600evb**

```
$(SDK)/meta-insyde/meta-evb/meta-evb-ast2600/recipes-platform/
```

G4:

- For Intel **BeechnutCity / AvenueCity** CRBs

```
$(SDK)/meta-insyde/meta-intel/meta-bhs/recipes-platform/configuration/
```

- For Nvidia **Oberon** CRB

```
$(SDK)/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/configuration/
```

- For **ast2600evb**

```
${SDK}/meta-insyde/meta-evb/meta-ast2600evb/recipes-platform/configuration/
```

- For **ast2700evb**

```
${SDK}/meta-insyde/meta-evb/meta-ast2700evb/recipes-platform/configuration/
```

New platform created by customer:

- **new platform**

```
$(SDK)/meta-base/meta-${new_platform}/recipes-platform/configuration/
```

The following sections will use the example of 'adding a PDB FRU to NV-starship, bus-address = 24-0x52, and assigning FRU ID = 3'.

3.2.3.1. Add the mapping between FRU ID and I2Cbus-address

First, add config json file to the entity-manager, path:

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/configuration/entity-manager/insyde_overlay/configurations/PDB_FRU.json
```

To edit the content, please refer to the following format:

- Field descriptions in Exposes:
    - Bus, Address: The bus and address of the EEPROM. If it is in variable form, its value corresponds to the bus and address of the FRU device in the Probe field.
    - Name: The property displayed in the Entity Manager D-Bus service.
    - FruSize: The maximum number of bytes for reading FRU data. If not set, the default value is 512.
    - FruID: The customized FRU ID is mapped to the bus and address.
    - Type: Fixed to "EEPROM".
- Name: Layer of Entity Manager object path.

- Probe: "xyz.openbmc_project.FruDevice({'BUS': 24, 'ADDRESS': 82})" represents that I2C device located at this bus-address. If the conditions in the Probe are met, the content of Expose field will be pushed to the Entity Manager D-Bus service.

- Type: Layer of Entity Manager object path.

```
{
    "Exposes": [
        {
            "Address": "$address",
            "Bus": "$bus",
            "Name": "$bus PDB_FRU",
            "FruSize": "256",
            "FruID": "3",
            "Type": "EEPROM"
        }
    ],
    "Name": "PDB",
    "Probe": "xyz.openbmc_project.FruDevice({'BUS': 24, 'ADDRESS': 82})",
    "Type": "Board",
    ...
}
```

Finally, add config json file to the entity-manager, recipe path:

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/configuration/entity-manager_%.bbappend
```

Added recipe content:

```
...
do_install:append:starship() {
    ...
    install -m 0444 ${S}/configurations/PDB_Board.json ${D}/usr/share/entity-manager/configurations/PDB_Board.json
    ...
}
```

3.2.3.2. Add FRU SDR

Add FRU SDR to Entity Manager, usually in the baseboard FRU config file (optional):

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/configuration/entity-manager/insyde_overlay/configurations/plat-NV-Starship/Grace_Hopper_P4351.json
```

Please refer to the IPMI Specification v2.0 (43.8 SDR Type 11h - FRU Device Locator Record) for the detailed format.

The following is a description of the necessary fields:

- A02SDRVersion: Fixed to 0x51
- A03RecordType: Fixed to 0x11, which means the SDR is FRU
- A04RecordLength: Represents the SDR length (from "AccessAddr" to "Name")
- SlaveAddr: Represents FRU ID
- EntityID: Represents Entity ID
- EntityInstance: Represents Entity Instance
- TypeLengthCode: Represents the length of Name, please refer to IPMI spec “43.15 Type/Length Byte Format”

- Name: Represents the name displayed by the FRU Table
- Type: Fixed to “IpmiSensor”

Fields other than the above do not affect the FRU Table but are necessary.

```
            {
                "A01RecordID": "0x0001",
                "A02SDRVersion": "0x51",
                "A03RecordType": "0x11",
                "A04RecordLength": "0x14",
                "AccessAddr": "0x20",
                "SlaveAddr": "0x03",
                "BusID": "0x80",
                "ChanNumber": "0x00",
                "Reserved": "0x00",
                "DevType": "0x10",
                "DevModifier": "0x00",
                "EntityID": "0x07",
                "EntityInstance": "0x01",
                "OEM": "0x00",
                "TypeLengthCode": "0xC9",
                "Name": "PDB_D FRU",
                "Type": "IpmiSensor"
            },
```

3.2.3.3. Verification

Conform the mapping between FRU ID and bus-address in Entity Manager using the following command:

```
busctl introspect xyz.openbmc_project.EntityManager <FRU config object path>
```

```
root@BMCXXXXXXXXXXXX:~# busctl introspect xyz.openbmc_project.EntityManager /xyz/openbmc_project/inventory/system/board/PDB/24_PDB_FRU
NAME                                    TYPE      SIGNATURE RESULT/VALUE FLAGS
org.freedesktop.DBus.Introspectable     interface -         -            -
.Introspect                             method    -         s            -
org.freedesktop.DBus.Peer               interface -         -            -
.GetMachineId                           method    -         s            -
.Ping                                   method    -         -            -
org.freedesktop.DBus.Properties         interface -         -            -
.Get                                    method    ss        v            -
.GetAll                                 method    s         a{sv}        -
.Set                                    method    ssv       -            -
.PropertiesChanged                      signal    sa{sv}as  -            -
xyz.openbmc_project.Configuration.EEPROM interface -         -            -
.Address                                property  t         82           emits-change
.Bus                                    property  t         24           emits-change
.FruID                                  property  t         3            emits-change
.FruSize                                property  t         256          emits-change
.Name                                   property  s         "24 PDB_FRU" emits-change
.Type                                   property  s         "EEPROM"     emits-change
root@BMCXXXXXXXXXXXX:~#
```

Figure 4-11. Mapping between FRU ID and bus-address in Entity Manager

Confirm whether the FRU SDR has been successfully added:

```
ipmitool sdr elist fru
```

```
root@BMCXXXXXXXXXXXX:~# ipmitool sdr elist fru
BMC FRU          | 00h | ns  |  7.1 | Logical FRU @00h
Baseboard_0      | 00h | ns  |  7.6 | Logical FRU @06h
M2_1             | 00h | ns  |  7.7 | Logical FRU @07h
M2_2             | 00h | ns  |  7.8 | Logical FRU @08h
Midplane FRU     | 00h | ns  |  7.5 | Logical FRU @04h
PDB_D FRU        | 00h | ns  |  7.4 | Logical FRU @03h
PSU 1 FRU        | 00h | ns  |  7.2 | Logical FRU @01h
PSU 2 FRU        | 00h | ns  |  7.3 | Logical FRU @02h
ProcMod_0        | 00h | ns  |  7.5 | Logical FRU @05h
U2_1             | 00h | ns  |  7.9 | Logical FRU @09h
U2_2             | 00h | ns  |  7.16 | Logical FRU @0Ah
root@BMCXXXXXXXXXXXX:~#
```

Figure 4-12. ipmitool sdr elist fru command

Use the ipmitool command to confirm whether the PDB FRU data is displayed successfully:

```
ipmitool raw 0xa 0x11 <FRU ID> <Offset LS Byte> <Offset MS Byte> <read bytes>
```

```
root@BMCXXXXXXXXXXXX:~# ipmitool raw 0x0a 0x11 0x00 0x00 0x00 0xff
 ff 01 00 00 01 00 00 00 fe 01 11 19 04 a5 dd c5
 32 30 30 34 6c cb 50 34 33 35 32 2f 50 34 33 35
 33 cd 54 48 41 4b 4e 32 32 34 38 30 30 30 53 d2
 37 35 33 2d 32 34 33 35 32 2d 30 30 30 35 2d 30
 30 30 d5 46 2d 44 43 2d 53 43 4d 5f 55 36 30 5f
 44 56 54 2e 6a 73 6f 6e d6 4d 41 43 31 3a 34 38
 3a 62 30 3a 32 64 3a 31 39 3a 31 31 3a 31 31 d7
 4d 41 43 32 3a 20 34 38 3a 42 30 3a 32 44 3a 31
 31 3a 31 31 3a 37 37 c3 42 2e 33 c1 00 00 00 00
 b7 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00 00
root@BMCXXXXXXXXXXXX:~#
```

Figure 4-13. ipmitool fru read command

Conform whether the "FRU Device Description" in the FRU Table is consistent with the "Name" in the newly added FRU SDR:

```
ipmitool fru print
```

```
FRU Device Description : PDB_D FRU (ID 3)
 Board Mfg Date        : Mon Aug 14 07:00:00 2023 UTC
 Board Mfg             : NVIDIA
 Board Product         : P4629/P4628
 Board Serial          : 1234567890123
 Board Part Number     : 753-24628-0002-000
 Board Extra           : B.3
 Board Area Checksum   : OK
```

Figure 4-14. ipmitool fru print command to verify FRU table

4. PEF & Alert Configuration

4.1. Introduction

The PEF (Platform Event Filtering) function monitors system events and can trigger alerts, power control actions, or logging operations based on predefined filters and policies.

4.2. Configuration

PEF settings can be highly detailed. It is recommended that users refer to the IPMI Specification v2.0 and configure settings based on the examples provided in this chapter.

The following is the path to the settings file of the default PEF setting:

- The configuration file of CRBs in **OPF G3** SDK:
    - For **Intel ArcherCity** CRB

```
${G3_SDK}/meta-insyde/meta-intel/meta-archercity/recipes-platform/ipmidcfg/files/plat/default/config.xml
```

    - For Nvidia **Falcon** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-Falcon/default/config.xml
```

    - For Nvidia **Starship** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-NV-Starship/default/config.xml
```

    - For Nvidia **MGX C2** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-Legoc2/default/config.xml
```

    - For Nvidia **MGX CG1** CRB

```
${G3_SDK}/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmidcfg/files/plat-Legocg1/default/config.xml
```

    - For **AMD Onyx / Quartz** CRBs

```
${G3_SDK}/meta-insyde/meta-amd/meta-sp5/recipe-platform/ipmidcfg/files/plat/default/config.xml
```

    - For **AMD Turin** CRB

```
${G3_SDK}/meta-insyde/meta-amd/meta-turin/recipes-platform/ipmidcfg/files/plat/default/config.xml
```

    - For **ast2600evb**

```
${G3_SDK}/meta-insyde/meta-evb/meta-evb-ast2600/recipes-platform/ipmidcfg/files/plat/default/config.xml
```

- The configuration file of CRBs in **OPF G4** SDK:
    - For Intel **BeechnutCity** CRBs

```
${G4_SDK}/meta-insyde/meta-intel/meta-bhs/recipes-platform/ipmidcfg/files/plat/beechnut_city/config.xml
```

    - For Intel **AvenueCity** CRBs

```
${G4_SDK}/meta-insyde/meta-intel/meta-bhs/recipes-
platform/ipmidcfg/files/plat/avenue_city/config.xml
```

- For Nvidia **Oberon** CRB

```
${G4_SDK}/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/ipmidcfg/files/plat-
general/gb200nvl-bmc-insyde/config.xml
```

- For **ast2600evb**

```
$(G4_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-
platform/ipmidcfg/files/plat/ast2600-evb-insyde/config.xml
```

- For **ast2700evb**

```
$(G4_SDK)/meta-insyde/meta-evb/meta-ast2700evb/recipes-
platform/ipmidcfg/files/plat/ast2600-evb-insyde/config.xml
```

- The configuration file of **new platform** created by customer:
  - For **new platform**

```
$(SDK)/meta-base/meta-${new_platform}/recipes-
platform/ipmidcfg/files/plat/$(SKU)/config.xml
```

These values will be loaded during the first time of the BMC initialization or after factory reset.

4.2.1. How to configure default PEF alert function

To pre-set the PEF function, user must ensure that the following settings are correct:

- PEF General Configuration
  - Set the enable/disable and basic behavior of the global PEF feature.
  - Define whether filtering and alerting are enabled and how triggered events are handled.
- Event Filter Table
  - Event Filter Table defines specific event filter conditions.
  - When an event occurs, the system determines whether to trigger an alert or other action based on filter matching conditions.
- Alert Policy Table
  - Alert Policy Table defines the alert actions and notification policies that should be taken when an event is triggered.
  - Specify the destination, channel, and alarm type to send the alert to.
- Alert configuration (choose one)
  - LAN Alert (SNMP Trap)
  - SMTP Alert Email

4.2.2. Example 1 (Threshold sensor and alert with Lan)

Create a PEF, monitor sensor number = 98h, when any event of "Upper Non-Critical - going high", "Upper Critical - going high" or "Upper Non-Recoverable - going high" occurs, PEF action will be executed: send alert trap to specific IP address and host power off.

4.2.2.1. Configuration in config.xml

- PEF Configuration:

(Refer to: IPMI Specification v2.0 "Table 30-, PEF Configuration Parameters": #1 PEF control and #2 PEF Action global control)

  - u8PEFControl: "Enable PEF" and "Enable event messages for PEF actions"
  - u8PEFActionGlobalControl: "Enable Alert Action"

```
<moduleconfig>
 <pefconfig u32Key="PEF_ID"
 u8PEFControl="0x03"
 u8PEFActionGlobalControl="0x01"
 u32SysEventSensorId="System Event"/>
 </moduleconfig>
```

- Event Filter Table:

(Refer to: IPMI Specification v2.0 "Table 17-, Event Filter Table Entry")

  - u8FilterConfiguration: enable filter
  - u8EventFilterAction: alert and power off
  - u8All: alert policy number, policy number = 1
  - u8EventSeverity: event severity, critical condition
  - u8GeneratorID1: match any
  - u8GeneratorID2: match any
  - u8SensorType: 01h = Temperature
  - u8SensorNumber: 98h
  - u8EventTriggerType: Threshold event
  - u16EventData1EventOffsetMask: 0x0a80

```
<moduleconfig>
 <pef_instance u32Key="PEF_PRESET_FILTERS_ID"
 u8FilterConfiguration="0x80"
 u8EventFilterAction="0x03"
 u8All="0x01"
 u8EventSeverity="0x10"
 u8GeneratorID1="0xff"
 u8GeneratorID2="0xff"
 u8SensorType="0x01"
 u8SensorNumber="0x98"
 u8EventTriggerType="0x01"
```

```
 u16EventData1EventOffsetMask="0x0a80"
 u8EventData1ANDMask="0x0"
 u8EventData1Compare1="0x0"
 u8EventData1Compare2="0x0"
 u8EventData2ANDMask="0x0"
 u8EventData2Compare1="0x0"
 u8EventData2Compare2="0x0"
 u8EventData3ANDMask="0x0"
 u8EventData3Compare1="0x0"
 u8EventData3Compare2="0x0"/>
 ...
 </moduleconfig>
```

- Alert Policy Table:

(Refer to: IPMI Specification v2.0 ”Table 17-, Alert Policy Table Entry”)

  - u8Policy: policy number = 1, entry is enabled, policy is “always send alert to this destination”
  - u8ChDestID: channel number = 1, destination selector = 1
  - u8AlertStringKey: 0x00 means no alert string key

```
<moduleconfig>
 <alert_policy_instance u32Key="ALERT_POLICY_ID"
 u8Policy="0x18"
 u8ChDestID="0x11"
 u8AlertStringKey="0x00"/>
 </moduleconfig>
```

- Alert configuration - LAN Destination:

(Refer to: IPMI Specification v2.0 “Table 23-, LAN Configuration Parameters”: #18 Destination Type and #19 Destination Addresses)

Hint: LAN Destination ID should match the u8ChDestID of the Alert Policy Table, for example, when u8ChDestID=0x11, u8LANChannelNumber of LAN Destination must be 1, and set in the first “landestconfig”.

  - u8LANDestAlertType:
    - 00h: PET trap, please select it when configure LAN alert.
    - 06h: OEM1, means SMTP alert email.
  - u8LANDestAlertAck:
    - 00h: Unacknowledged, alert is assumed successful if transmission occurs.
    - 01h: Acknowledged, alert is assumed successful only if acknowledged is returned.
  - u8LANDestAlertTimeout: 0x05 means 5 seconds.
  - u8LANDestAlertRetries: 0x02 means 2 time to retry. (Maximum retries are 0x07)
  - u8LANDestAlertGWSelector:
    - 00h: Default Gateway
    - 01h: Backup Gateway

- u8LANDestAlertAddrFormat:
    - 00h: IPv4
    - 01h: IPv6
- u8LANDestIPMAC: IP address (To comply with the IPMI command format, the 0x00 after the IP address cannot be omitted.)

```
<moduleconfig>
 ...
 <alertdestconfig u32Key="ALERT_DEST_ID"
                  u8LANChannelNumber="1">
 <!-- The number of landestconfig depend on LAN_DEST_COUNT -->
 <landestconfig u32Key="LAN_DEST_ID"
 u8LANDestAlertType="0x00"
 u8LANDestAlertAck="0x01"
 u8LANDestAlertTimeout="0x05"
 u8LANDestAlertRetries="0x02"
 u8LANDestAlertGWSelector="0x00"
 u8LANDestAlertAddrFormat="0x00"
 u8LANDestIPMAC="0xc0 0xa8 0x09 0xc2 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00" />
 ...
 <landestconfig u32Key="LAN_DEST_ID"
 u8LANDestAlertType="0x00"
 u8LANDestAlertAck="0x00"
 u8LANDestAlertTimeout="0x00"
 u8LANDestAlertRetries="0x00"
 u8LANDestAlertGWSelector="0x00"
 u8LANDestAlertAddrFormat="0x00"
 u8LANDestIPMAC="0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00"
 ...
 </alertdestconfig>
 ...
 </moduleconfig>
```

- Alert configuration - SNMP Trap:
    - u8CommunityString: Authentication string, when v3 is selected as SNMP Version, this field has no effect.
    - u8SNMPTrapVersion:
        - 00h: SNMPv1
        - 01h: SNMPv2c
        - 03h: SNMPv3 (Provides authentication and encryption)
    - u8SNMPTrapUserName: set to “test”
    - u8SNMPTrapSecLevel:
        - 01h: AuthNoPriv (Authentication, no encryption)
        - 03h: AuthPriv (Authentication and encryption)
    - u8SNMPTrapAuthProtocol:

- 01h: MD5
- 02h: SHA
- 03h: SHA224
- 04h: SHA256
- 05h: SHA284
- 06h: SHA512

- u8SNMPTrapAuthPassphrase: set to “Test123456”
- u8SNMPTrapPrivProtocol:
  - 01h: AES
- u8SNMPTrapPrivPassphrase: set to “Test123456”

```
<lanchannelconfig u32Key="LAN_ID"
u8LANChannelNumber="1"
...
u8CommunityString="public"
...
u8SNMPTrapVersion="SNMP_TRAP_V3"
u8SNMPTrapUserName="test"
u8SNMPTrapSecLevel="0x03”
u8SNMPTrapAuthProtocol="0x04"
u8SNMPTrapAuthPassphrase="Test123456"
u8SNMPTrapPrivProtocol="0x01"
u8SNMPTrapPrivPassphrase="Test123456"
/>
```

4.2.2.2. Verification

- PEF Global Configuration verification:

```
ipmitool pef status -v
```

```
Last SEL addition        : 12/11/24 03:18:02 UTC
Last SEL record ID       : 0x000d
Last S/W processed ID    : 0x0000
Last BMC processed ID    : 0x000d
PEF                      : enabled
PEF event messages       : enabled
PEF startup delay        : disabled
Alert startup delay      : disabled
Alert                    : active
Power-off                : inactive
Reset                    : inactive
Power-cycle              : inactive
OEM-defined              : inactive
Diagnostic-interrupt     : inactive
```

Figure 5-1. Verify the PEF Global Configuration for example 1

- Event Filter Table verification:

```
ipmitool pef filter list -v
```

Hint: Event Filter Table entry is added according to the setting order of config.xml

```
PEF Filter Table entry   : 12
Status                   : enabled, configurable
Sensor type              : Temperature
Sensor number            : 152
Event severity           : Critical
Event class              : Threshold
Event trigger(s)         : (0x01/0x0a80),<UNC,<UC,<UNR
Action                   : Alert,Power-off
Policy set               : 1
```

Figure 5-2. Verify the Event Filter Table for example 1

- Alert Policy Table& LAN Destination verification:

```
ipmitool pef policy list -v
```

```
Alert policy table entry : 1
Policy set               : 1
State                    : enabled
Policy entry rule        : Match-always
Channel number           : 1
Channel medium           : 802.3 LAN
Alert destination type   : PET
PET Community            : public
ACK timeout/retry (secs) : 5
Retries                  : 2
IP address               : 192.168.9.194
MAC address              : 00:00:00:00:00:00
```

Figure 5-3. Verify the Alert Policy Table for example 1

Test the PEF function and confirm whether the set LAN Alert (SNMP trap) is received:

- Disable other temperature-related Event Filter Tables to avoid affecting the test
- Trigger PEF through "Add SEL Entry" Command:

```
ipmitool raw 0xa 0x44 0x0 0x0 0x2 0xf9 0x0b 0xa6 0x61 0x41 0x0 0x4 0x1 0x98 0x01 0x09
0xff 0xff
```

- Open wireshark on the Alert receiving end and confirm whether SNMP trap is received
- Since we set SNMP trap v3, the data is encrypted and needs to be authenticated, so the encryption status will be displayed as shown in Figure 5-4:

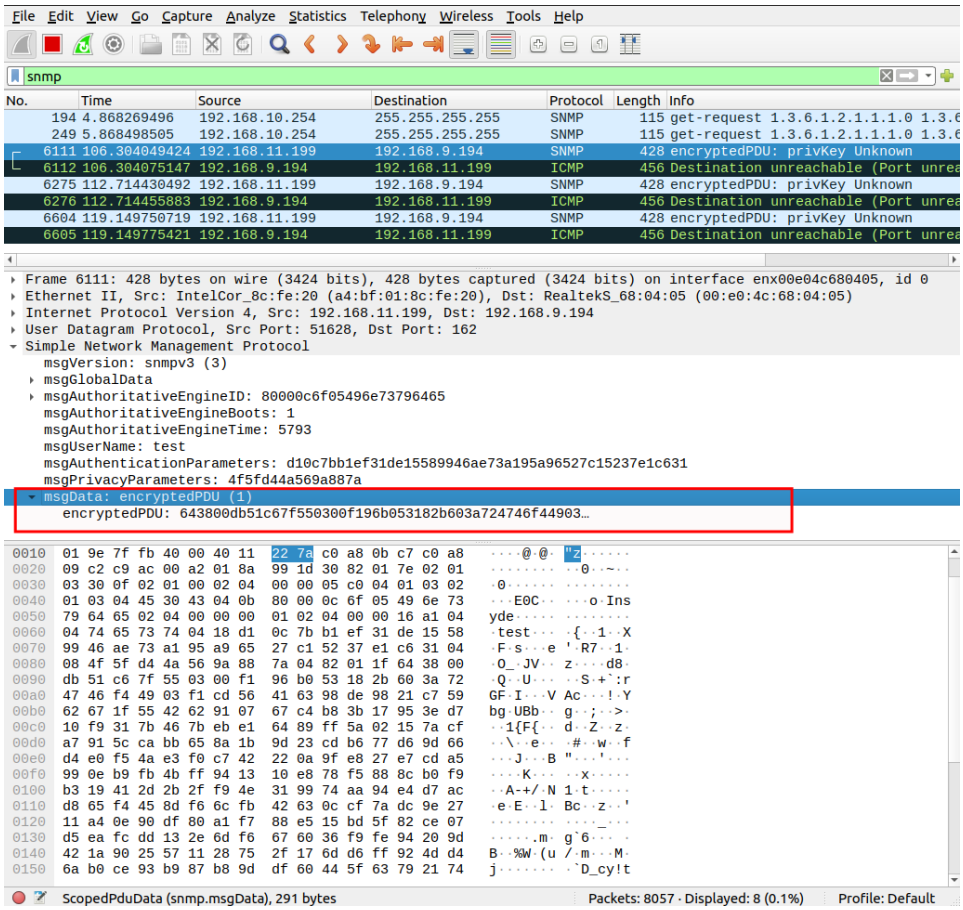


Figure 5-4. Received encrypted SNMP trap in wireshark

To decrypt the SNMP trap, find the SNMP settings in Wireshark and enter the following information, as shown in Figure 5-5:

- Engine ID = n/a
- Username = test
- Authentication model = SHA-256
- Password = Test123456
- Privacy protocol = AES
- Privacy password = Test123456

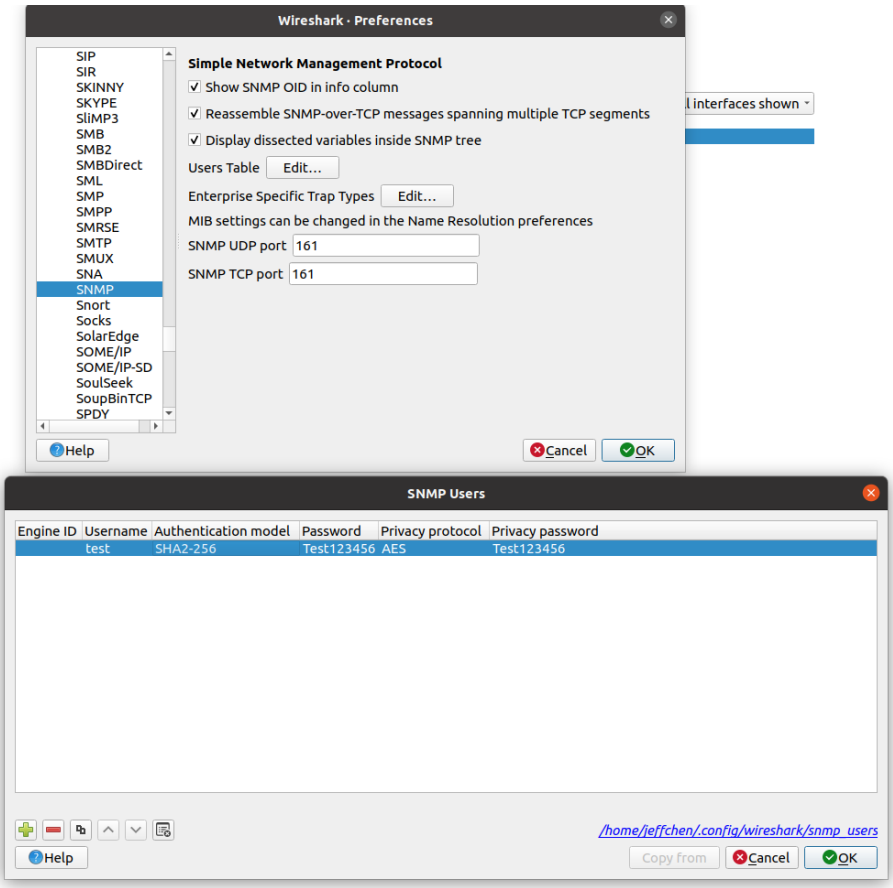


Figure 5-5. Wireshark settings to parse SNMP Trap v3

After saving the settings, the users can successfully see the decrypted content of the received SNMP Trap, as shown in Figure 5-6:

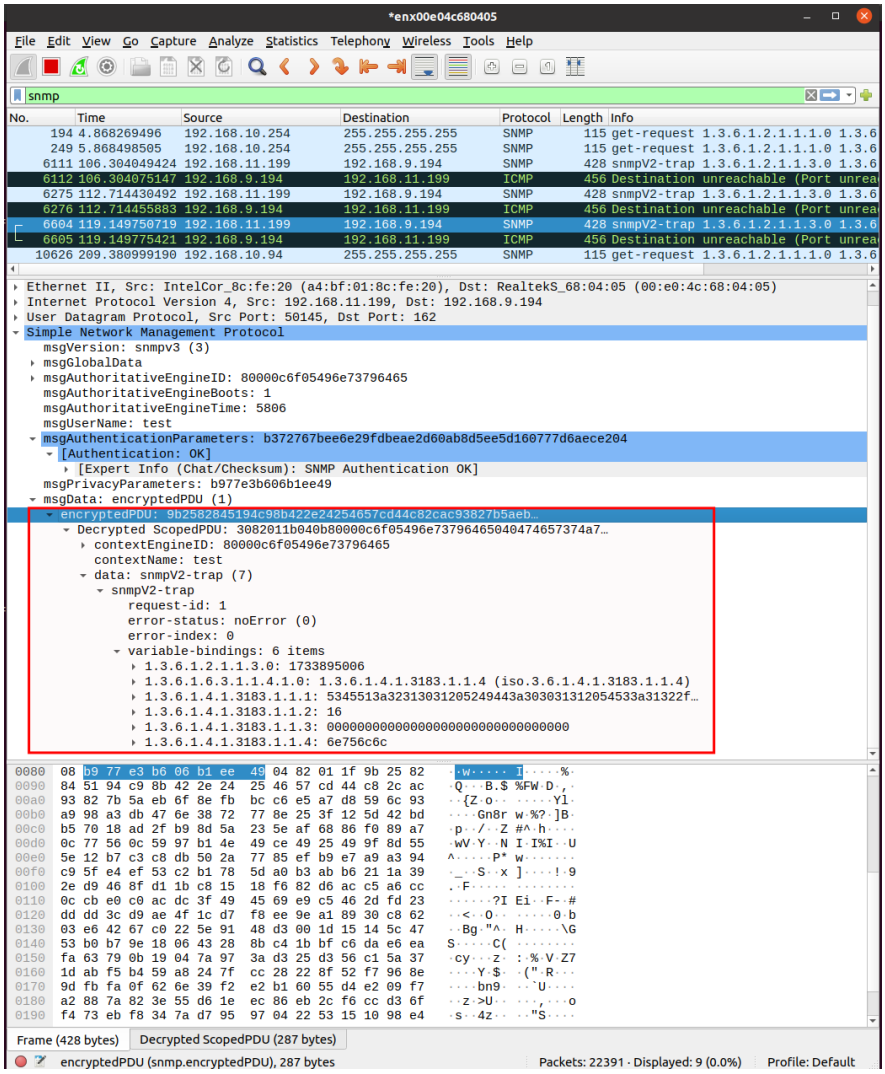


Figure 5-6. Wireshark successfully parses SNMP Trap v3

4.2.3. Example 2 (Discrete sensor and alert with SMTP mail)

Create a PEF, monitor a sensor with sensor type "Logging disabled", the number is 0xFD, PEF will be triggered when Clear SEL occurs, and the PEF action is to send SMTP alert mail.

More detailed settings are provided below.

4.2.3.1. Configuration in config.xml

- PEF Configuration:

(Refer to: IPMI Specification v2.0 “Table 30-, PEF Configuration Parameters”: #1 PEF control and #2 PEF Action global control)

  - `u8PEFControl`: "Enable PEF" and "Enable event messages for PEF actions"
  - `u8PEFActionGlobalControl`: "Enable Alert Action"

```
<moduleconfig>
 <pefconfig u32Key="PEF_ID"
 u8PEFControl="0x03"
 u8PEFActionGlobalControl="0x01"
 u32SysEventSensorId="System Event"/>
 </moduleconfig>
```

- Event Filter Table:

(Refer to: IPMI Specification v2.0 "Table 30-, PEF Configuration Parameters": #1 PEF control and #2 PEF Action global control)

  - u8FilterConfiguration: enable filter.
  - u8EventFilterAction: only alert
  - u8All: alert policy number - policy number 2
  - u8EventSeverity: event severity - Unspecified
  - u8GeneratorID1: match any
  - u8GeneratorID2: match any
  - u8SensorType: 10h = Logging disabled
  - u8SensorNumber: FDh
  - u8EventTriggerType: event/reading type, Generic Discrete event
  - u16EventData1EventOffsetMask: 04 00, select bit2: Log Area Reset/Cleared

```
<moduleconfig>
 <pef_instance u32Key="PEF_PRESET_FILTERS_ID"
 u8FilterConfiguration="0x80"
 u8EventFilterAction="0x01"
 u8All="0x02"
 u8EventSeverity="0x00"
 u8GeneratorID1="0xff"
 u8GeneratorID2="0xff"
 u8SensorType="0x10"
 u8SensorNumber="0xfd"
 u8EventTriggerType="0x6f"
 u16EventData1EventOffsetMask="0x0004"
 u8EventData1ANDMask="0x0"
 u8EventData1Compare1="0x0"
 u8EventData1Compare2="0x0"
 u8EventData2ANDMask="0x0"
 u8EventData2Compare1="0x0"
 u8EventData2Compare2="0x0"
 u8EventData3ANDMask="0x0"
 u8EventData3Compare1="0x0"
 u8EventData3Compare2="0x0"/>
 ...
 </moduleconfig>
```

- Alert Policy Table:

(Refer to: IPMI Specification v2.0 "Table 17-, Event Filter Table Entry")

- u8Policy: policy number = 2, entry is enabled, policy is "always send alert to this destination"
- u8ChDestID: channel number = 1, Destination selector = 2
- u8AlertStringKey: 0x00 means no alert string key

```
<moduleconfig>
 <alert_policy_instance u32Key="ALERT_POLICY_ID"
 u8Policy="0x28"
 u8ChDestID="0x12"
 u8AlertStringKey="0x00"/>
 </moduleconfig>
```

- Alert configuration – LAN Destination:

(Refer to: IPMI Specification v2.0 "Table 23-, LAN Configuration Parameters": #18 Destination Type and #19 Destination Addresses)

- **Hint: LAN Destination ID should match the u8ChDestID of the Alert Policy Table, Example: When u8ChDestID=0x12, u8LANChannelNumber of LAN Destination must be 1, and set in the second "landestconfig".**
  - u8LANDestAlertType: 0x06 means SMTP. (When using smtp alert email, the other fields have no effect)

```
<moduleconfig>
 ...
 <alertdestconfig u32Key="ALERT_DEST_ID"
                  u8LANChannelNumber="1">
 <!-- The number of landestconfig depend on LAN_DEST_COUNT -->
 <landestconfig u32Key="LAN_DEST_ID"
 u8LANDestAlertType="0x00"
 u8LANDestAlertAck="0x00"
 u8LANDestAlertTimeout="0x00"
 u8LANDestAlertRetries="0x00"
 u8LANDestAlertGWSelector="0x00"
 u8LANDestAlertAddrFormat="0x00"
 u8LANDestIPMAC="0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00" />
 <landestconfig u32Key="LAN_DEST_ID"
 u8LANDestAlertType="0x06"                          <<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<<
 u8LANDestAlertAck="0x00"
 u8LANDestAlertTimeout="0x00"
 u8LANDestAlertRetries="0x00"
 u8LANDestAlertGWSelector="0x00"
 u8LANDestAlertAddrFormat="0x00"
 u8LANDestIPMAC="0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00 0x00
0x00 0x00"
 ...
 </alertdestconfig>
 ...
 </moduleconfig>
```

- Alert configuration (SMTP alert email)
  - u8SMTPServerDomainName: SMTP server domain or IP address
  - u16SMTPServerPort: SMTP server port, 587 for gmail (support STARTTLS)

- u8SMTPAuthMethod:
  - 00h: Off
  - 01h: Auto (automatically select the most appropriate authentication method for authentication)
  - 03h: Plain
  - 04h: MD5
  - 06h: Login
- u8SMTPFromUser: Sender email address, set to “bmc.robot.insyde@gmail.com”
- u8SMTPAuthUsername: Authentication account, set to “bmc.robot.insyde@gmail.com”
- u8SMTPAuthPassword: Authentication password, set to <password>
- u8SMTPTlsStarttlsEn:
  - 00h: StartTLS disable
  - 01h: StartTLS enable (Public Mail Services must be enabled)
- u8SMTPTlsEn:
  - 00h: TLS/SSL disable
  - 01h: TLS/SSL enable (Public Mail Services must be enabled)

```
<lanchannelconfig u32Key="LAN_ID"
...
u8SMTPServerDomainName="smtp.gmail.com"
u16SMTPServerPort="587"
u8SMTPAuthMethod="0x01"
u8SMTPFromUser="sender@gmail.com"
u8SMTPAuthUsername="sender@gmail.com"
u8SMTPAuthPassword="password"
u8SMTPTlsStarttlsEn="0x01"
u8SMTPTlsEn="0x01"
...
/>
```

- `u8EmailAddr`: set “bmc.smtp.test@gmail.com” to receiver email address.
- `u8MsgSubject`: set “Alert test” to email subject.
- `u8MsgBody`: set “This is alert” to email body.

```
<moduleconfig>
 ..........
 ..........
 <alertdestconfig ...>
 ..........
 <smtpdestconfig u32Key="SMTP_DEST_ID"
 u8EmailAddr="bmc.smtp.test@gmail.com"
 u8MsgSubject="test"
 u8MsgBody="Alert" />
 ..........
 </alertdestconfig>
```

```
..........
..........
</moduleconfig>
```

4.2.3.2. Verification

- PEF Global Configuration verify:

```
ipmitool pef status -v
```

```
Last SEL addition        : 12/11/24 03:18:02 UTC
Last SEL record ID       : 0x000d
Last S/W processed ID    : 0x0000
Last BMC processed ID    : 0x000d
PEF                      : enabled
PEF event messages       : enabled
PEF startup delay        : disabled
Alert startup delay      : disabled
Alert                    : active
Power-off                : inactive
Reset                    : inactive
Power-cycle              : inactive
OEM-defined              : inactive
Diagnostic-interrupt     : inactive
```

Figure 5-7. Verify the PEF Global Configuration for example 2

- Event Filter Table verify:

```
ipmitool pef filter list -v
```

Event Filter Table Entry is added according to the setting order of config.xml

```
PEF Filter Table entry   : 12
Status                   : enabled, configurable
Sensor type              : Logging disabled
Sensor number            : 253
Event severity           : None
Event class              : Discrete
Event trigger(s)         : Sensor-specific
Action                   : Alert
Policy set               : 2
```

Figure 5-8. Verify the Event Filter Table for example 2

- Alert Policy Table and LAN Destination verify:

```
ipmitool pef policy list -v
```

(OEM1 means “SMTP destination type”)

```
Alert policy table entry : 2
Policy set               : 2
State                    : enabled
Policy entry rule        : Match-always
Channel number           : 1
Channel medium           : 802.3 LAN
Alert destination type   : OEM 1
ACK timeout/retry (secs) : 0
Retries                  : 0
IP address               : 0.0.0.0
MAC address              : 00:00:00:00:00:00
```

Figure 5-9. Verify the Alert Policy Table for example 2

- Check SMTP configuration on bmcweb, as shown in Figure 5-10:
  - Bmcweb > Settings > Alert email

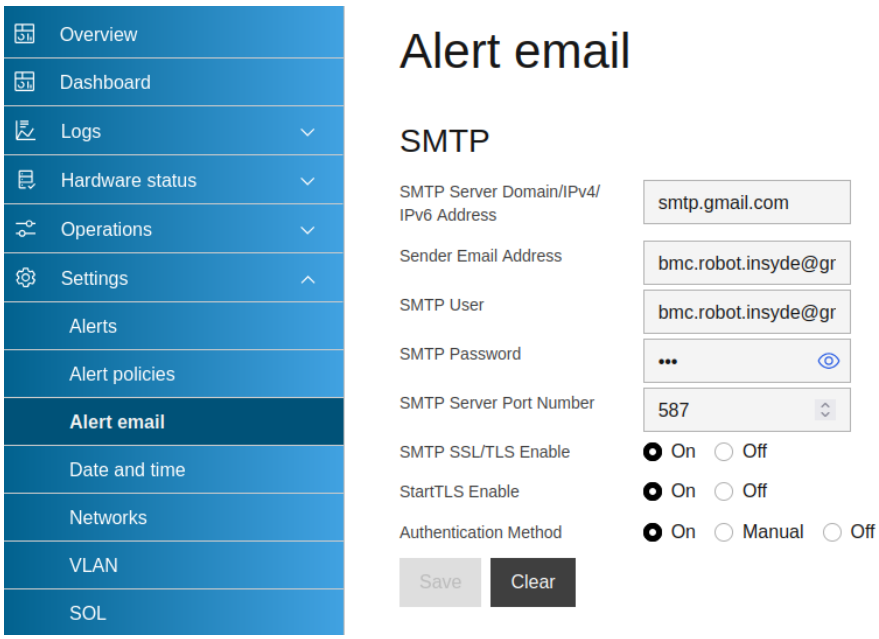


Figure 5-10. SMTP configuration in bmcweb

- Check SMTP Receiver email address in bmcweb, as shown in Figure 5-11:
  - Bmcweb > Settings > Alerts

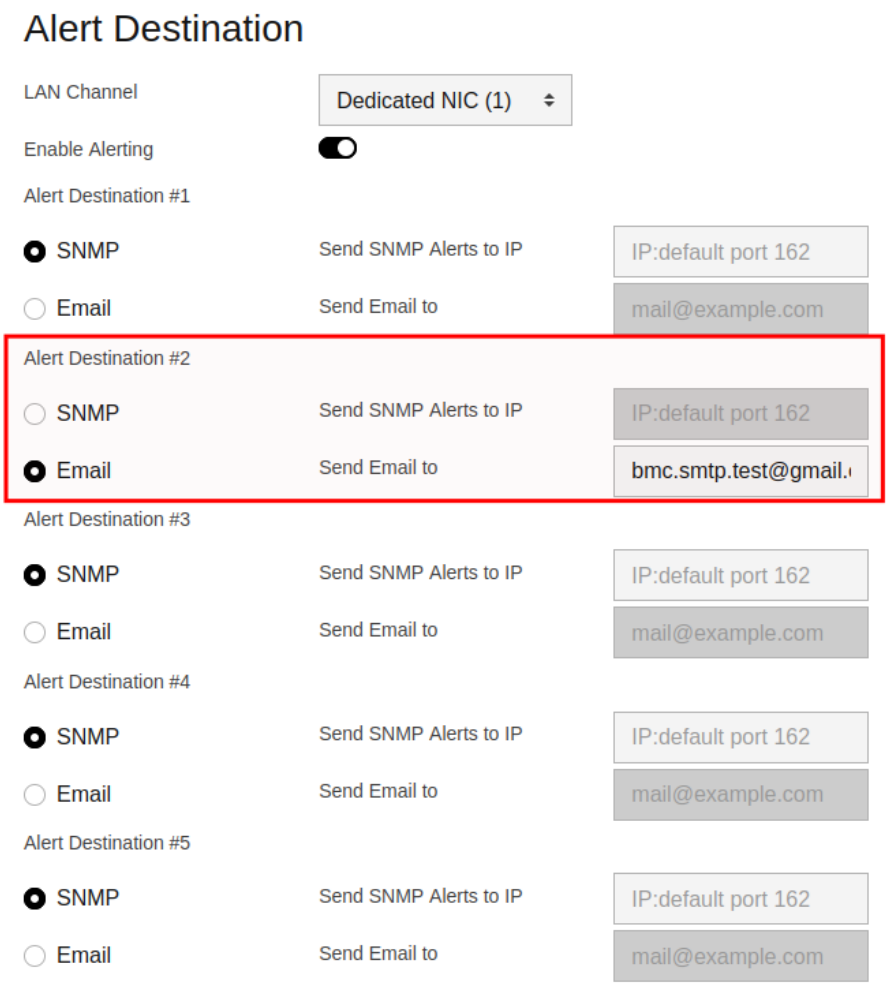


Figure 5-11. SMTP receiver email address in bmcweb

Try to trigger PEF with following command:

```
ipmitool sel clear
```

Check whether the destination successfully received the alert email:

Figure 5-12. Successfully received the SMTP alert email

5. Device ID Configuration

5.1. Introduction

This page describes how to configure response data of the IPMI Get Device ID command (NetFn=App, CMD=01h), including firmware revision, Device ID, Device Revision, Manufacturer ID, Product ID & Auxiliary Firmware Revision. More information in **IPMI Specification v2.0 section 20.1 Get Device ID Command**

5.2. Configuration

5.2.1. Firmware Revision & Auxiliary Firmware Revision

Configuration field description:

Name	Size（byte）	Get Device ID Response Data byte #	Description
Firmware Rev1	1	4	Firmware revision 1 [7] Device available:0=normal operation, 1= device firmware, SDR Repository update or self-initialization in progress. [Firmware / SDR Repository updates can be differentiated by issuing a Get SDR command and checking the completion code.] [6:0] Major Firmware Revision, binary encoded.
Firmware Rev2	1	5	Firmware Revision 2: Minor Firmware Revision. BCD encoded.
Auxiliary Firmware Rev	4	13:16	Auxiliary Firmware Revision Information. This field is optional. If present, it holds additional information about the firmware revision, such as boot block or internal data structure version numbers. The meanings of the numbers are specific to the vendor identified by Manufacturer ID (see below). When the vendor-specific definition is not known, generic utilities should display each byte as 2-digit hexadecimal numbers, with byte 13 displayed first as the most- significant byte.

The configuration file path depends on vendor and target platform:

- The configuration file of CRBs in **OPF G3** SDK:
  - For Intel **ArcherCity** CRB

```
$(OPF_SDK)/meta-insyde/meta-intel/recipes-core/os-release/os-release.bbappend
```

  - For AMD **Onyx** / **Quartz** / **Turin** CRBs

```
$(OPF_SDK)/meta-insyde/meta-amd/recipes-core/os-release/os-release.bbappend
```

  - For Nvidia **Falcon** / **Starship** / **MGX CG1** / **MGX C2** CRBs

```
$(OPF_SDK)/meta-insyde/meta-nvidia/recipes-core/os-release/os-release.bbappend
```

  - For **ast2600evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-evb-ast2600/recipes-core/os-release/os-
release.bbappend
```

- The configuration file of CRBs in **OPF G4** SDK:
  - For Intel **BeechnutCity** / **AvenueCity** CRBs

```
$(OPF_SDK)/meta-insyde/meta-intel/recipes-core/os-release/os-release.bbappend
```

  - For Nvidia **Oberon** CRB

```
$(OPF_SDK)/meta-insyde/meta-nvidia/recipes-core/os-release/os-release.bbappend
```

  - For **ast2600evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-core/os-release/os-
release.bbappend
```

  - For **ast2700evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2700evb/recipes-core/os-release/os-
release.bbappend
```

- The configuration file of **new platform** created by customer:

```
$(OPF_SDK)/meta-base/meta-${new_platform}/recipes-platform/os-release/os-
release.bbappend
```

Below is an example with following settings:

- Firmware revision Major = 25
- Firmware revision Minor = 01
- Auxiliary Firmware Revision = 0x00 **0x05 0x00** 0x30
  - Note: first byte always be 0x00
  - byte 2,3 is LSB with **0005** = 0x05 0x00

Configuration in file:

```
OPF_VERSION = "25.01.30.0005"
```

See the result at below **Verification** section

5.2.2. Device ID, Device Revision, Additional Device Support, Manufacturer ID & Product ID

Configuration field description:

Name	Size（byte）	Get Device ID Response Data byte #	Description
Device ID	1	2	00h = unspecified.
Device Rev	1	3	Device Revision [7] 1= device provides Device SDRs 0 = device does not provide Device SDRs [6:4] reserved. Return as 0. [3:0] Device Revision, binary encoded.
Additional Device Support	1	7	Additional Device Support (formerly called IPM Device Support). Lists the IPMI 'logical device' commands and functions that the controller supports that are in addition to the mandatory IPM and Application commands. [7] Chassis Device （device functions as chassis device per ICMB spec.） [6] Bridge (device responds to Bridge NetFn commands) [5] IPMB Event Generator (device generates event messages [platform event request messages] onto the IPMB) [4] IPMB Event Receiver (device accepts event messages [platform event request messages] from the IPMB) [3] FRU Inventory Device [2] SEL Device [1] SDR Repository Device [0] Sensor Device
Manufacturer ID	3	8:10	Manufacturer ID, LS Byte first. The manufacturer ID is a 20-bit value that is derived from the IANA 'Private Enterprise' ID (see below). Most significant four bits = reserved (0000b). 000000h = unspecified. 0FFFFFh = reserved. This value is binary encoded. E.g. the ID for the IPMI forum is 7154 decimals, which is 1BF2h, which would be stored in this record as F2h, 1Bh, 00h for bytes 8 through 10, respectively.
Product ID	2	11:12	Product ID, LS Byte first. This field can be used to provide a number that identifies a particular system, module, add-in card, or board set. The number is specified according to the manufacturer given by Manufacturer ID (see below). 0000h = unspecified. FFFFh = reserved.

The configuration file path depends on vendor and target platform:

- The configuration file of CRBs in **OPF G3** SDK:
    - For Intel **ArcherCity** CRB

```
$(OPF_SDK)/meta-insyde/meta-intel/meta-archercity/recipes-platform/ipmi/phosphor-ipmi-config/dev_id.json
```

- For AMD **Onyx** / **Quartz** CRBs

```
$(OPF_SDK)/meta-insyde/meta-amd/meta-sp5/recipes-platform/ipmi/phosphor-ipmi-config/dev_id.json
```

- For AMD **Turin** CRB

```
$(OPF_SDK)/meta-insyde/meta-amd/meta-turin/recipes-platform/ipmi/phosphor-ipmi-config/dev_id.json
```

- For Nvidia **Falcon** CRB

```
$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmi/phosphor-ipmi-config/dev_id-Falcon.json
```

- For Nvidia **Starship** CRB

```
$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmi/phosphor-ipmi-config/dev_id-NV-Starship.json
```

- For Nvidia **MGX CG1** CRB

```
$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmi/phosphor-ipmi-config/dev_id-Legocg1.jso
```

- For Nvidia **MGX C2** CRB

```
$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gh/recipes-platform/ipmi/phosphor-ipmi-config/dev_id-Legoc2.json
```

- For **ast2600evb**

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-evb-ast2600/recipes-platform/ipmi/phosphor-ipmi-config/dev_id.json
```

- The configuration file of CRBs in **OPF G4** SDK:
  - For Intel **BeechnutCity** / **AvenueCity** CRBs

```
$(OPF_SDK)/meta-intel/meta-bhs/recipes-platform/ipmi/phosphor-ipmi-config/dev_id.json
```

  - For Nvidia **Oberon** CRB

```
$(OPF_SDK)/meta-insyde/meta-nvidia/meta-gb200nvl/recipes-platform/ipmi/phosphor-ipmi-config/dev_id-general.json
```

  - For ast2600evb

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2600evb/recipes-platform/ipmi/phosphor-ipmi-config/dev_id.json
```

  - For ast2700evb

```
$(OPF_SDK)/meta-insyde/meta-evb/meta-ast2700evb/recipes-core/os-release/os-release.bbappend
```

- The configuration file of **new platform** created by customer:

```
    $(OPF_SDK)/meta-base/meta-${new_platform}/recipes-platform/ipmi/phosphor-ipmi-
config/dev_id.json
```

Below is an example with following settings:

- Device ID = 0x22 = **34** in Decimal
- Device Rev = 0x81 = **129** in Decimal
- Additional Device Support = 0x9f = **159** in Decimal
- Manufacturer ID = 0x09 0xb0 0x00 = 0xb009 in LSB = **45065** in Decimal
- Product ID = 0x02 0x00 = 0x02 in LSB = **2** in Decimal
- **Note**: "aux" is useless here.

Configuration in file:

```
{"id": 34, "revision": 129, "addn_dev_support": 159,
    "manuf_id": 45065, "prod_id": 2, "aux": 0}
```

See the result at below **Verification** section

5.3. Verification

Verify the configuration by using IPMI command **mc info** via lanplus

```
ipmitool -U root -P OpenBmc -I lanplus -C 17 -H 192.168.59.151 mc info
Device ID                 : 34
Device Revision           : 1
Firmware Revision         : 25.01
IPMI Version              : 2.0
Manufacturer ID           : 45065
Manufacturer Name         : Insyde
Product ID                : 2 (0x0002)
Product Name              : Unknown (0x2)
Device Available          : yes
Provides Device SDRs      : yes
Additional Device Support :
    Sensor Device
    SDR Repository Device
    SEL Device
    FRU Inventory Device
    IPMB Event Receiver
    Chassis Device
Aux Firmware Rev Info     :
    0x00
    0x05
    0x00
    0x30
```

Note:

- Device ID = 0x22 in hexadecimal = **34** in Decimal
- Manufacturer Name is paring by IANA Private Enterprise Numbers (PENs)
    - Manufacturer Name is **Insyde** in example
    - More detail in https://www.iana.org/assignments/enterprise-numbers/?page=451
- Additional Device Support = **159** in Decimal = 0x9f in Hex = 0x10011111 in Binary
    - See the Additional Device Support at previous field description